



Université du Québec
à Chicoutimi

ESSAI

PRÉSENTÉ À

L'UNIVERSITÉ DU QUÉBEC À CHICOUTIMI

COMME EXIGENCE PARTIELLE

DE LA MAÎTRISE EN INFORMATIQUE

PAR

KAMIL SALEH

**INTÉGRATION DU RÉFÉRENTIEL MITRE ATT&CK
DANS UN ÉCOSYSTÈME MCP**

SEPTEMBRE 2026



RÉSUMÉ

Dans un centre d'opérations de sécurité, le goulot d'étranglement n'est plus la détection mais la qualification de ce qui est détecté. Rattacher une activité observée au référentiel MITRE ATT&CK reste un travail manuel, lent et tributaire d'une expertise rare, et la difficulté redouble chez un exploitant d'infrastructure énergétique, dont les incidents traversent la frontière entre informatique de gestion et informatique industrielle.

Cet essai conçoit et mesure un écosystème agentique ancrant un modèle de langage sur ATT&CK au moyen du *Model Context Protocol*. Un corpus de 42 échantillons est produit en laboratoire, à l'abri de la contamination qui fausse les bancs d'essai publics, sur deux zones — systèmes d'information et automates — et deux matrices, Enterprise et ICS. Dix modèles de la génération courante y sont évalués sans ancrage, en trois exécutions par échantillon, soit 1 260 appels, selon une méthodologie qui sépare cinq catégories de prédiction et deux bases de calcul. Le modèle retenu est ensuite rejoué avec le serveur, à corpus et conditions identiques.

Sans ancrage, aucun modèle ne dépasse 0,603 de F1 au niveau technique, et la performance chute d'un tiers sur la matrice industrielle. Les identifiants fabriqués sont exclusivement des tactiques, et exclusivement sur des journaux industriels : l'erreur est un défaut de repérage entre univers, non une invention arbitraire. Avec ancrage, le F1 technique passe de 0,563 à 0,706, la matrice industrielle double presque, et le taux d'identifiants périmés ne diminue pas : il disparaît. La contribution ne tient pas à l'accès à une source, mais à quatre décisions de conception — version épinglée et jointe à chaque réponse, résolution des révocations, granularité d'outils bornée, trace d'usage vérifiable.

Mots-clés : MITRE ATT&CK, Model Context Protocol, grands modèles de langage, ancrage, centre d'opérations de sécurité, technologies opérationnelles, ICS, évaluation.

TABLE DES MATIÈRES

Résumé	i
Table des matières	iii
Table des figures	v
Liste des tableaux	vii
Liste des abréviations et sigles	ix
Remerciements	1
Introduction générale	2
1 Préparation	4
1.1 Revue de littérature	4
1.1.1 Axe 1 — Les LLM en cyberdéfense : valeur, mesure, limites	4
1.1.2 Axe 2 — Ancrer par la récupération : le RAG et ses limites	7
1.1.3 Axe 3 — Ancrer par les outils : de l'apprentissage par outils au MCP	8
1.1.4 Axe 4 — Le revers : sécurité des agents outillés et du MCP	13
1.1.5 Synthèse : acquis et lacunes	17
1.2 Méthodologie d'évaluation	18
1.2.1 Construction du corpus	19
1.2.2 Étiquetage des prédictions	24
1.2.3 Calcul des métriques	28
1.2.4 Sélection des modèles	31
1.2.5 Évaluation et synthèse	33
2 L'approche MCP	46
2.1 Introduction	46
2.2 Le serveur MCP	46
2.2.1 Architecture : hôte, client et serveur	46
2.2.2 Les deux sources du serveur	46
2.2.3 Le chemin d'une requête	47

2.2.4	Le moteur de mapping	53
2.3	Évaluation de l'approche	59
2.3.1	La ligne à battre	59
2.3.2	Ce que la phase 2 ajoute au protocole	60
2.3.3	Les indicateurs ajoutés	61
2.3.4	Ce que la phase 2 doit montrer	63
2.3.5	Résultats	63
2.4	Les avantages de l'approche	66
Conclusion générale		69
Bibliographie		71

TABLE DES FIGURES

1.1	L'arc de l'ancrage, fil conducteur des axes 1 à 3 : chaque étape corrige la précédente et en déplace la limite. L'essai porte sur la troisième étape et sur sa comparaison empirique avec la deuxième.	5
1.2	Deux façons d'ancrer le même raisonnement. Le RAG récupère un voisin plausible par similarité ; l'accès outillé récupère la définition exacte par une requête structurée. La surface d'attaque ne disparaît pas : elle se déplace de la base vers la chaîne d'approvisionnement du serveur.	8
1.3	Architecture du MCP. L'hôte instancie un client par serveur ; chaque serveur expose outils, ressources et invites sur JSON-RPC 2.0. Le serveur du bas est le connecteur SIEM de l'essai. Tout ce qui est à droite de la frontière est, par défaut, non fiable : c'est le point de départ de l'axe 4.	10
1.4	Surface d'attaque d'un serveur MCP le long de son cycle de vie (taxonomie adaptée de Hou et al. [1], chiffres de MCPTox [2] et des CVE [3, 4]) et réponses de conception retenues pour le serveur de l'essai.	15
1.5	Génération de la télémétrie TO et TI.	20
1.6	Hiérarchie à trois niveaux du référentiel, illustrée par la technique T1059. . .	25
2.1	Architecture MCP. Le modèle raisonne dans l'hôte ; le client porte la connexion ; le serveur répond à partir de ses deux sources, en lecture seule. . .	46
2.2	Le référentiel MITRE tel que le serveur le lit : des objets reliés par des relations explicites.	47
2.3	Chemin d'une requête dans le serveur MCP.	48
2.4	L'énumération des outils : le modèle lit là les dix-huit descriptions, figées dans le code, qui guident ensuite son choix d'outil.	49
2.5	Ligne d'audit écrite pour cet appel. En bleu, l'identifiant qui corréle la session ; en vert, la durée et la version qui datent la réponse ; en ocre, les arguments tronqués ; en rouge, le résumé du résultat.	53
2.6	Le moteur de mapping : proposer (chaîne du haut), puis vérifier et confronter (chaîne du bas), avec le rejet comme sortie légitime.	53
2.7	Cas d'utilisation : chaîne conversationnelle n8n. L'agent reçoit la question, dispose du serveur MCP MITRE comme outil, et chaque appel est journalisé avant le retour de la réponse.	68

LISTE DES TABLEAUX

1.1	Révisions de la spécification MCP et implications de sécurité [5–7].	11
1.2	Ce que la littérature empirique établit sur le MCP (axe 3).	12
1.3	Mesures de la vulnérabilité des agents outillés et du MCP (axe 4).	16
1.4	Ce que la revue établit et ce qu'elle laisse ouvert, par question de revue. . . .	18
1.5	Composition du corpus de la vérité de terrain.	20
1.6	Vérité de terrain par échantillon : tactique, technique et sous-technique atten- dues.	24
1.7	Étiquetage du niveau sous-technique.	27
1.8	Ce qui change d'une base à l'autre.	29
1.9	Application numérique dans les deux bases.	30
1.10	Modèles évalués, éditeurs et voies d'accès.	32
1.11	Étiquetage des prédictions des dix modèles. Entre parenthèses, le nombre d'exécutions évaluées au niveau considéré. FP est la valeur de la base stricte : elle comprend les colonnes OBS et H, reportées à part comme sous-comptes ; FN = FP + dont Ab.	35
1.12	Classement de la qualité du mapping, base stricte.	37
1.13	Classement de la qualité du mapping, base tolérante.	39
1.14	Répartition des identifiants obsolètes par matrice et taux d'obsolescence, par modèle.	41
1.15	Classement de la fiabilité, du temps de traitement et du coût.	43
2.1	Les dix-huit outils exposés et leur description, telle que le modèle la lit. . . .	50
2.2	Claude Opus 4.8 sans serveur : la ligne à battre.	59
2.3	Les indicateurs ajoutés, leur formule et leur unité de compte.	62
2.4	Étiquetage des prédictions, sans puis avec serveur. $N = VP + FP + H + O +$ A, vérifié sur les six lignes.	64
2.5	Qualité du mapping, base stricte. Ent. et ICS : F1 technique ventilé par matrice. .	64
2.6	Qualité du mapping, base tolérante. Écart obs. = F1 tolérant - F1 strict, au niveau technique.	64
2.7	Les échantillons du corpus exposés à une révocation, avant et après ancrage. .	65
2.8	Usage du serveur, campagne avec MCP.	65

2.9	Croisement, au niveau technique, entre ce que le serveur a rendu et ce que le modèle a retenu.	66
2.10	Fiabilité, vitesse et coût.	66

LISTE DES ABRÉVIATIONS ET SIGLES

Ab	Abstention : refus explicite du modèle de proposer un identifiant
AAIF	<i>Agentic AI Foundation</i> , fonds dirigé de la Linux Foundation
API	<i>Application Programming Interface</i> — interface de programmation
ATT&CK	<i>Adversarial Tactics, Techniques and Common Knowledge</i> , base de connaissances des comportements adverses publiée par MITRE
CISA	<i>Cybersecurity and Infrastructure Security Agency</i> (États-Unis)
CTI	<i>Cyber Threat Intelligence</i> — renseignement sur les menaces
CVE	<i>Common Vulnerabilities and Exposures</i> , identifiant public d'une vulnérabilité
CVSS	<i>Common Vulnerability Scoring System</i> , échelle de gravité d'une vulnérabilité
CWE	<i>Common Weakness Enumeration</i> , catalogue des classes de faiblesses logicielles
F1	Score F1 : moyenne harmonique de la précision et du rappel, $F_1 = 2PR/(P + R)$
FN	Faux négatif : la réponse attendue n'a pas été retrouvée
FP	Faux positif : un identifiant est proposé, mais il est incorrect
GRFICS	<i>Graphical Realism Framework for Industrial Control Simulation</i> , simulateur de procédé industriel
H	Hallucination : identifiant inexistant dans le référentiel
HTTP	<i>HyperText Transfer Protocol</i>
IA	Intelligence artificielle
ICS	<i>Industrial Control System</i> — système de contrôle industriel
IT	<i>Information Technology</i> (équivalent anglais de TI)
JSON	<i>JavaScript Object Notation</i> , format d'échange de données
JSON-RPC	Protocole d'appel de procédure à distance encodé en JSON
LLM	<i>Large Language Model</i> — grand modèle de langage
LSASS	<i>Local Security Authority Subsystem Service</i> , processus Windows détenant les identifiants en mémoire
MCP	<i>Model Context Protocol</i> , protocole ouvert normalisant l'accès d'une application LLM à des outils et à des données
MITRE	<i>The MITRE Corporation</i> , organisme sans but lucratif éditeur d'ATT&CK (le nom n'est pas un sigle)
MM	Mauvaise matrice : identifiant pris hors de la matrice ATT&CK considérée
OAuth	<i>Open Authorization</i> , cadre de délégation d'autorisation

OBS	Identifiant obsolète : identifiant retiré du référentiel (<i>deprecated</i> ou <i>revoked</i>)
OT	<i>Operational Technology</i> (équivalent anglais de TO)
OWASP	<i>Open Worldwide Application Security Project</i>
PLC	<i>Programmable Logic Controller</i> — automate programmable industriel
QR	Question de recherche (QR1 à QR4)
RAG	<i>Retrieval-Augmented Generation</i> — génération augmentée par récupération
RFC	<i>Request for Comments</i> , document normatif de l'IETF
SCADA	<i>Supervisory Control and Data Acquisition</i> , système de supervision industrielle
SIEM	<i>Security Information and Event Management</i> , plateforme de collecte et de corrélation des journaux de sécurité
SOC	<i>Security Operations Center</i> — centre des opérations de sécurité
SSE	<i>Server-Sent Events</i> , transport unidirectionnel sur HTTP
STIX	<i>Structured Threat Information eXpression</i> , format normalisé de publication du renseignement sur les menaces
TI	Technologies de l'information
TO	Technologies opérationnelles
TRAM	<i>Threat Report ATT&CK Mapper</i> , outil de classification supervisée du Center for Threat-Informed Defense
USB-C	<i>Universal Serial Bus Type-C</i>
VP	Vrai positif : l'identifiant proposé correspond à la vérité de terrain

REMERCIEMENTS

Je tiens à remercier en premier lieu mon directeur d'essai, Kevin Bouchard, ainsi que Fehmi Jaafar, pour leur encadrement, leur disponibilité et la rigueur de leurs conseils tout au long de ce travail. Leurs relectures successives et leurs questions ont orienté ce projet autant qu'elles l'ont corrigé.

Mes remerciements vont également à Franck Jeannot, mon superviseur, dont l'accompagnement en contexte professionnel a permis d'ancrer cette recherche dans des préoccupations opérationnelles réelles.

Je remercie le laboratoire Cynergia et l'ensemble de son équipe pour l'accès aux ressources et aux environnements de test, sans lesquels la campagne expérimentale présentée dans cet essai n'aurait pas été possible, ainsi que pour les échanges qui l'ont nourrie.

Enfin, je remercie ma famille et mes proches pour leur soutien constant.

INTRODUCTION GÉNÉRALE

Pendant longtemps, défendre un système d'information a consisté à en garder la porte. On admet aujourd'hui que l'intrusion finira par se produire, et que la valeur d'une défense se mesure à sa capacité de la détecter, de la comprendre et de la contenir pendant qu'elle a lieu. Au sein d'un centre d'opérations de sécurité, deux équipes sur trois déclarent ne plus suivre le rythme, et les analystes juniors se renouvellent plus vite qu'ils ne se forment [8]. La recherche a donné un nom à ce phénomène, la *fatigue d'alerte* [9], et un diagnostic : le goulot d'étranglement n'est pas la détection, mais la qualification de ce qui est détecté.

Le référentiel MITRE ATT&CK [10] est devenu la langue commune de cet exercice ; y cartographier une activité observée reste pourtant un travail manuel, lent, et tributaire d'une expertise rare. Pour un exploitant d'infrastructure énergétique, la difficulté redouble : l'incident type traverse la frontière entre l'informatique de gestion et l'informatique industrielle (automates, capteurs, protocoles de terrain) deux mondes qui ne parlent ni les mêmes journaux ni la même matrice ATT&CK, et dont la convergence est précisément ce que visent les campagnes les plus avancées.

Les grands modèles de langage (LLM) semblent taillés pour cette tâche. Mais on ne peut pas encore leur faire confiance : ils *hallucinent* [11], et leur savoir est en outre figé à la date de leur entraînement, quand le référentiel, lui, évolue à chaque version. La solution ce n'est pas un modèle plus grand mais un modèle *ancré*, rattaché au moment de répondre à une source vérifiable et à jour plutôt qu'à sa seule mémoire. Le *Model Context Protocol* (MCP), ouvert fin 2024 et confié depuis à une gouvernance neutre [5, 12], standardise cet ancrage en donnant au modèle un accès structuré, déterministe et journalisable à des outils et à des données.

C'est dans ce contexte que s'inscrit le présent essai, qui conçoit un écosystème agentique intégrant le MCP et MITRE ATT&CK pour détecter et contextualiser les attaques. Quatre questions le guident :

1. **Q1 — Où en sont les modèles sans ancrage ?** Dans quelle mesure les modèles de langage actuels savent-ils rattacher au référentiel une activité malveillante lue dans des journaux bruts, aux trois niveaux de la hiérarchie (tactique, technique, sous-technique) ? Cette capacité se maintient-elle lorsque l'on passe de la matrice Entreprise à la matrice ICS, c'est-à-dire du journal de système d'information au journal d'automate ? La réponse suppose un corpus produit en laboratoire, à l'abri de la contamination qui fausse les bancs d'essai publics.
2. **Q2 — Que change l'ancrage déterministe ?** À corpus, consigne et conditions d'ap-

pel identiques, que gagne un modèle qui interroge un référentiel avant de répondre par rapport au même modèle répondant de mémoire ? Le gain attendu porte sur trois défauts distincts : les identifiants fabriqués, les identifiants périmés, et la confusion entre les deux matrices. Encore faut-il les mesurer séparément.

3. **Q3 — Comment construire le serveur qui le permet ?** Quelles propriétés un serveur MCP doit-il réunir pour que cet ancrage soit exact, reproductible et daté : quelle granularité d'outils, quelle version du référentiel, quel traitement des identifiants révoqués, quelle trace jointe à chaque réponse ? La littérature fixe des conditions d'emploi, elle ne les éprouve pas sur cette tâche.
4. **Q4 — À quel coût en surface d'attaque ?** Ancrer un modèle déplace le risque du texte vers l'action : le serveur devient un canal d'instruction autant qu'une source de faits. Quelles garanties de conception — lecture seule, moindre privilège, journalisation, validation des entrées — rendent ce déplacement acceptable pour un exploitant d'infrastructure essentielle soumis à la LPCE ?

CHAPITRE 1

PRÉPARATION

Ce premier chapitre établit le cadre théorique et opérationnel de notre étude. Nous proposons d’abord une revue de littérature qui examine la capacité des approches actuelles et par la suite nous présenterons la méthodologie d’évaluation retenu pour valoriser l’apport de notre approche.

1.1 REVUE DE LITTÉRATURE

Cette revue se lit comme un entonnoir : on étudie les grands modèles de langage (LLM) en cyberdéfense l’ancrage par la récupération, l’ancrage par les outils jusqu’au MCP, et puis on traite le revers de la sécurité des systèmes agentiques.

1.1.1 Axe 1 — Les LLM en cyberdéfense : valeur, mesure, limites

Les synthèses récentes attestent d’usages allant de la détection d’intrusion à l’assistance à l’analyste ; la vaste revue de Kheddar dans *Information Fusion* en fournit la cartographie pour les systèmes de détection d’intrusion [13]. L’axe examine successivement la tâche pivot de l’essai (le *mapping* ATT&CK), la façon de mesurer sans se tromper, la fiabilité, puis l’angle mort industriel.

Cartographier vers ATT&CK : deux générations et un plafond

MITRE ATT&CK est une base de connaissances ouverte qui catalogue les comportements adverses en *techniques* (p. ex. « vol d’identifiants en mémoire »), regroupées par *tactiques* (l’objectif poursuivi) ; l’édition Enterprise compte environ deux cents techniques et quatorze tactiques [10]. *Mapper* une menace consiste à lire une description d’activité — rapport d’incident, journal, règle de détection — et à retrouver la ou les techniques qu’elle illustre : c’est la brique de base de la détection contextualisée.

Génération 1 : la classification supervisée. L’outil emblématique est TRAM (*Threat Report ATT&CK Mapper*) du Center for Threat-Informed Defense [14] : un modèle SciBERT affiné sur des phrases étiquetées à la main ne couvre qu’une cinquantaine des techniques les plus fréquentes. Le plafond n’est pas paramétrique mais structurel : l’annotation experte est un goulot d’étranglement et la couverture reste partielle, comme le documente la synthèse de Rahman *et al.* sur l’extraction automatique de renseignement [15].

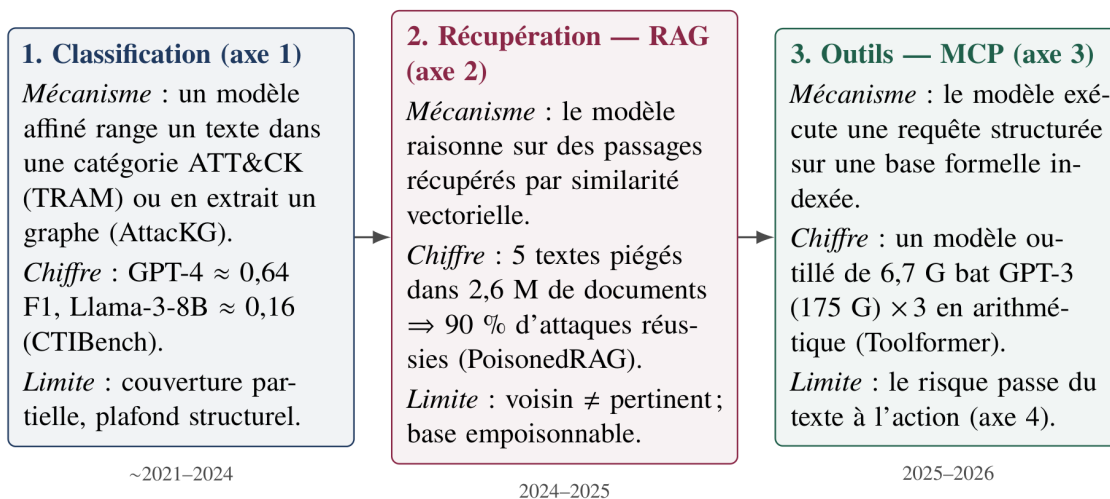


FIGURE 1.1 – L’arc de l’ancrage, fil conducteur des axes 1 à 3 : chaque étape corrige la précédente et en déplace la limite. L’essai porte sur la troisième étape et sur sa comparaison empirique avec la deuxième.

Génération 1 bis : les graphes de connaissances. AttacKG (ESORICS 2022) reconstruit, à partir d’un rapport, le graphe des actions de l’attaquant, puis agrège les graphes de plusieurs rapports en graphes de techniques [16]. Sur un large corpus réel il identifie 28 262 techniques et 8393 indicateurs de compromission ; sur seize rapports annotés, il atteint des F1 de 0,887 (entités), 0,896 (dépendances) et 0,789 (techniques), au-dessus des méthodes antérieures.

La mesure : CTIBench. Ce banc d’essai (NeurIPS 2024, piste *Datasets and Benchmarks*) évalue les LLM sur des tâches de renseignement, dont l’extraction de techniques (CTI-ATE) [17]. Les modèles ouverts y sont faibles (Llama-3-8B $\approx$ 0,16 de F1) ; les modèles propriétaires ne sont pas mûrs pour un usage non supervisé (ChatGPT-3.5 $\approx$ 0,31, ChatGPT-4 0,64, meilleur du panel)¹. Son prolongement AthenaBench montre que la domination des modèles propriétaires ne s’étend pas au raisonnement — attribution d’un attaquant, choix d’une mitigation — où tous restent faibles [18].

Conclusion : Classification ou graphe, la première génération bute sur un plafond qui tient à la nature de la tâche : la *mapping* n’est pas une reconnaissance de mots-clés mais un raisonnement contextuel, et même le meilleur LLM généraliste s’arrête à 0,64 de F1 sans ancrage (QR1).

1. Le papier est ambigu sur la métrique : le texte parle de micro-F1, le tableau de macro-F1. Les valeurs sont celles du tableau publié.

Mesurer sans se tromper : capacité et contamination

CyberSecEval (Meta) mesure deux propensions : produire du code vulnérable et se laisser enrôler dans une attaque, sur 8 langages, 50 faiblesses CWE et 10 catégories de techniques ATT&CK [19]. En moyenne les modèles suggèrent du code vulnérable dans environ 30 % des cas, et — motif récurrent de cette revue — les modèles les plus compétents sont souvent les plus enclins à l’insécurité.

La contamination des jeux de test est le piège méthodologique central. Si les exemples d’un banc d’essai, et surtout leurs réponses, figuraient dans les données d’entraînement, le modèle récite au lieu de raisonner. PrimeVul (ICSE 2025) le démontre sur la détection de vulnérabilités [20] : avec un jeu dédoublé et découpé chronologiquement (près de 7000 fonctions vulnérables, 228 800 saines, 140 CWE), un modèle de 7 milliards de paramètres crédité de 68,3 % de F1 sur BigVul s’effondre à 3,1 %, GPT-4 se rapproche du hasard, et 18,9 % des échantillons de BigVul fuyaient dans le jeu de test.

Conclusion : Les bancs d’essai publics surestiment massivement les performances parce que les modèles ont déjà vu les données. Ce résultat justifie directement, dans l’essai, la génération de journaux propres en laboratoire plutôt que la réutilisation d’un jeu public (QR1).

Fiabilité : l’ancrage plutôt que la taille

La synthèse de Ji *et al.* distingue l’hallucination intrinsèque (le modèle contredit la source fournie) de l’extrinsèque (il affirme l’invérifiable) [11]. En cybersécurité les conséquences sont concrètes : attribuer une technique inexistante oriente l’investigation puis la remédiation vers la mauvaise cible, au point que Mezzi, Massacci et Tuma concluent à l’*unreliability* des LLM pour le renseignement sur les menaces lorsqu’ils opèrent sans garde-fou [21]. Le point capital, et contre-intuitif, est que la fiabilité factuelle dépend de l’ancrage et non de la taille : le programme *Inverse Scaling* (TMLR 2023) a mis au jour des tâches où un modèle plus grand fait pire, notamment parce qu’il s’appuie avec excès sur ses connaissances mémorisées [22].

Sur le terrain, enfin, la contrainte première n’est pas la détection mais le volume : la synthèse de Tariq *et al.* sur la fatigue d’alerte (ACM CSUR 2025) décrit un déluge quotidien d’alertes majoritairement faussement positives, dont une part importante reste sans investigation [9]. Les LLM y ont une valeur d’augmentation — résumer une alerte, interpréter une commande, contextualiser un incident — avec un humain dans la boucle pour les cas critiques.

Conclusion : Ancrer le raisonnement dans une base vérifiable est plus efficace qu’augmenter la puissance de calcul ; c’est l’argument directeur de l’essai (QR1).

L’angle mort : la détection en environnement industriel (OT/ICS)

L’informatique de gestion (IT) et l’informatique industrielle (OT : automates, capteurs, protocoles de terrain comme Modbus qui pilotent des procédés physiques) sont deux mondes ; or, pour un opérateur d’infrastructure critique, l’incident type traverse la frontière IT/OT. La quasi-totalité des travaux ci-dessus porte sur l’IT. Les rares travaux industriels sont éclairants : Kong, Saber, Youssef et Kundur évaluent un LLM prêt à l’emploi comme couche de détection explicable et supervisée sur du trafic Modbus d’un réseau électrique, en insistant sur des alarmes auditable — comprises, tracées, archivées pour la réponse à incident [23] — un cahier des charges très proche de celui d’Hydro-Québec. La synthèse de Kheddar confirme que le modèle est surtout employé comme assistant sur des rapports en langage naturel, rarement comme analyste de jetons de protocole industriel [13].

Conclusion : L’évaluation des LLM sur des journaux OT réels reste largement inexplo-
rée : c’est le vide que le laboratoire IT/OT de l’essai comble (QR1).

1.1.2 Axe 2 — Ancrer par la récupération : le RAG et ses limites

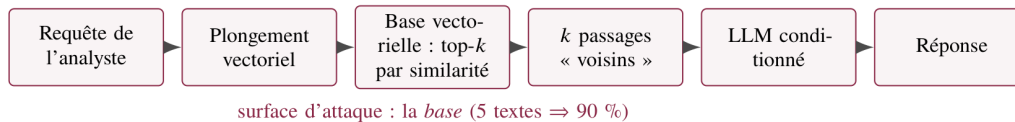
L’axe 1 a posé le problème : les LLM aident mais hallucinent, et leur fiabilité tient à l’ancrage. La communauté a exploré deux voies (figure 1.1) : récupérer de l’information, objet de cet axe, puis appeler des outils (axe 3).

Définition. La génération augmentée par récupération (RAG) a été formalisée par Lewis *et al.* (NeurIPS 2020) comme le couplage d’un récupérateur dense — qui plonge requête et documents dans un même espace vectoriel et sélectionne les k passages les plus proches — et d’un générateur conditionné sur ces passages [24]. Le modèle ne répond plus « de mémoire » mais sur la foi de documents fournis à la volée, ce qui réduit l’hallucination et permet de citer. La taxonomie de Gao *et al.* distingue le RAG naïf, avancé (reformulation, re-classement) et modulaire [25] ; les systèmes ci-dessous relèvent de la seconde catégorie.

Application au mapping. Technique RAG est l’état de l’art : pour un texte donné, il récupère des exemples de techniques pertinents, les fait re-classer par un LLM, puis génère la réponse avec un modèle affiné, surpassant nettement les approches sans ancrage [26]. RAM (*Rule-ATT&CK Mapper*) applique la même logique aux règles de détection d’un SIEM — le moteur central du SOC qui collecte les journaux et déclenche les alertes — et relie chaque règle aux techniques qu’elle détecte, aux composants de données et aux mitigations associés [27].

Trois limites. (a) *Similarité n'est pas pertinence* : le récupérateur renvoie ce qui ressemble à la requête, et un voisin vectoriel peut décrire une technique adjacente mais distincte, précisément là où le *mapping* demande une discrimination fine. (b) *Non-déterminisme* : le résultat dépend du modèle de plongement, du découpage des documents et de k ; deux exécutions ne sont pas garanties identiques, ce qui complique l'audit d'une alerte. (c) *La base est une surface d'attaque* : PoisonedRAG (USENIX Security 2025) formule l'empoisonnement comme un problème d'optimisation — le texte injecté doit être récupéré *et* induire la réponse voulue — et obtient 90 % de réussite, jusqu'à 97 % sur certains jeux, en injectant cinq textes dans une base de plus de 2,6 millions de documents ; les défenses testées (reformulation, détection par perplexité) sont insuffisantes [28]. La figure 1.2 oppose cette chaîne probabiliste à l'accès déterministe de l'axe 3.

RAG (axe 2) — probabiliste



Accès déterministe via MCP (axe 3)

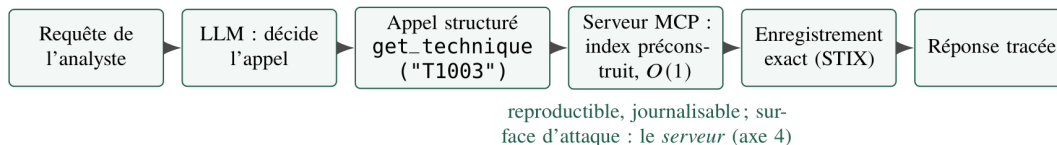


FIGURE 1.2 – Deux façons d’ancrer le même raisonnement. Le RAG récupère un voisin plausible par similarité ; l’accès outillé récupère la définition exacte par une requête structurée. La surface d’attaque ne disparaît pas : elle se déplace de la base vers la chaîne d’approvisionnement du serveur.

Conclusion : La récupération par ressemblance n’est ni pleinement fiable (un voisin peut être hors sujet, et le résultat n’est pas reproductible) ni pleinement sûre (la base est empoisonnable). D’où la recherche d’un ancrage plus déterministe (**QR2**).

1.1.3 Axe 3 — Ancrer par les outils : de l’apprentissage par outils au MCP

La seconde voie ne récupère pas un document ressemblant : elle appelle un outil qui renvoie un fait exact. C’est le paradigme de l’apprentissage par outils (*tool learning*), fondement théorique direct du MCP.

Le paradigme de l’apprentissage par outils

Toolformer (NeurIPS 2023) est un modèle qui a appris, de façon auto-supervisée, à décider lui-même quand insérer un appel d’outil au fil de sa génération : il insère des appels

candidats dans des textes, les exécute, et ne conserve que ceux dont le résultat réduit sa perplexité sur la suite [29]. Cinq outils sont testés (calculatrice, recherche Wikipédia, questions-réponses, traduction, calendrier) sur un modèle de 6,7 milliards de paramètres. Sur les problèmes arithmétiques, Toolformer atteint 29,4 % (SVAMP) et 40,4 % (ASDiv) sans exemple, contre 10,0 % et 14,0 % pour GPT-3 — 175 milliards de paramètres, soit un modèle environ 25 fois plus gros ; le contre-exemple est instructif : sur les questions factuelles, le moteur de recherche employé est trop rudimentaire et GPT-3 reprend l’avantage. L’outil ne vaut que ce que vaut sa source.

ReAct (ICLR 2023) est une méthode de sollicitation qui alterne explicitement « pensées » (raisonnement en langage naturel) et « actions » (appels d’outils) [30]. En ancrant chaque étape dans un résultat concret, ReAct réduit l’hallucination sur HotpotQA et, sur deux environnements interactifs (ALFWorld, WebShop), dépasse de 34 et 10 points de taux de succès des méthodes entraînées sur 10^3 à 10^5 exemples, avec un ou deux exemples seulement. La synthèse de Qin *et al.* (ACM CSUR 2024) généralise : doter un modèle d’outils atténue à la fois l’obsolescence de ses connaissances et ses hallucinations, et déplace le problème vers la sélection et la composition des outils [31].

Conclusion : Le bon appel d’outil vaut plus que vingt-cinq fois plus de paramètres — à condition que l’outil renvoie un fait exact (**QR3**).

Le MCP : standardiser l’ancrage par outils

Définition et architecture. Le *Model Context Protocol*, ouvert par Anthropic en novembre 2024, n’est pas un modèle mais un protocole qui standardise l’accès d’une application LLM à des outils et à des données [5]. L’analogie usuelle est le port USB-C : un connecteur unique remplace une multitude de câbles propriétaires. L’architecture (figure 1.3) distingue l’hôte (l’application), qui instancie un client par connexion, et les serveurs qui exposent trois primitives : des *outils* (fonctions invocables), des *ressources* (données en lecture) et des *invites* (gabarits) ; le client expose en retour *sampling*, *roots* et *elicitation*. Les messages sont en JSON-RPC 2.0 sur deux transports : *stdio* pour un serveur local et *Streamable HTTP* pour un serveur distant, ce dernier protégé par une autorisation OAuth 2.1 [6].

Évolution et gouvernance. Le protocole a mûri vite (tableau 1.1). La révision 2025-03-26 a ajouté l’autorisation OAuth 2.1, le transport *Streamable HTTP* et des annotations décrivant le comportement d’un outil (lecture seule, destructif) ; la révision 2025-06-18 a classé les serveurs comme *resource servers* OAuth, rendu obligatoires les indicateurs de ressource (RFC 8707) pour empêcher un serveur malveillant d’obtenir des jetons, et introduit la sortie structurée et l’*elicitation* [6]. Le 9 décembre 2025, Anthropic a transféré le protocole à

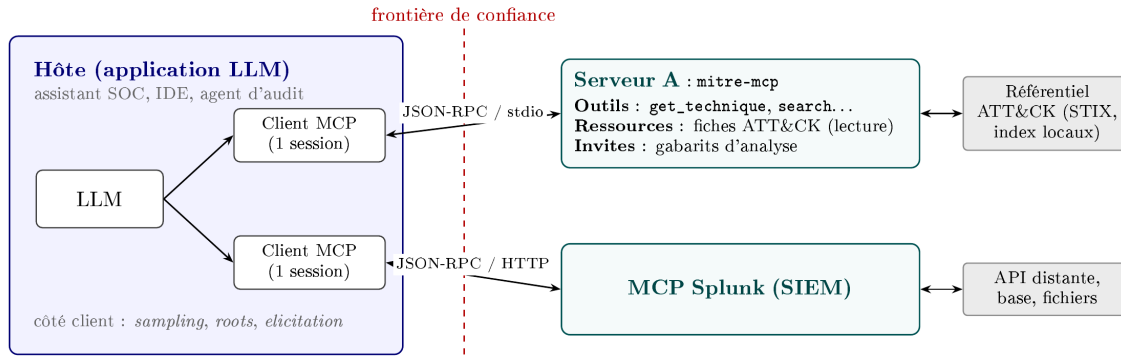


FIGURE 1.3 – Architecture du MCP. L’hôte instancie un client par serveur ; chaque serveur expose outils, ressources et invites sur JSON-RPC 2.0. Le serveur du bas est le connecteur SIEM de l’essai. Tout ce qui est à droite de la frontière est, par défaut, non fiable : c’est le point de départ de l’axe 4.

l’Agentic AI Foundation, fonds dirigé de la Linux Foundation cofondé avec Block et OpenAI, à un moment où plus de 10 000 serveurs publics étaient actifs [12]. Enfin la révision 2026-07-28 — la plus importante depuis le lancement — rend le noyau du protocole sans état, durcit l’autorisation (validation de l’émetteur, *Client ID Metadata Documents*), formalise un cadre d’extensions et une politique de dépréciation [7] : un serveur distant devient une charge HTTP ordinaire, à laquelle s’applique l’outillage de sécurité HTTP classique.

Révision	Apports majeurs	Implication pour un serveur de SOC
2024-11-05	Primitives (outils, ressources, invites) ; stdio et HTTP+SSE	Garde-fous de consentement en <i>should</i> ; sécurité laissée à l’implémenteur
2025-03-26	OAuth 2.1 ; <i>Streamable HTTP</i> ; annotations d’outils ; lots JSON-RPC	Les annotations sont des indications : la spécification les déclare non fiables sauf serveur de confiance
2025-06-18	Serveurs = <i>resource servers</i> ; RFC 8707 obligatoire ; sortie structurée ; <i>elicitation</i> ; page de bonnes pratiques	Premiers <i>must</i> de sécurité ; sortie typée = validation possible côté client
2025-11-25	Révision intermédiaire ; consolidation par SEP et groupes de travail	Gouvernance communautaire (AAIF)

Révision	Apports majeurs	Implication pour un serveur de SOC
2026-07-28	Noyau sans état ; autorisation durcie (émetteur, CIMD) ; cadre d’extensions ; politique de dépréciation	Serveur distant = charge HTTP standard ; extensions à auditer séparément

TABEAU 1.1 – Révisions de la spécification MCP et implications de sécurité [5–7].

Application au *mapping* : l’accès déterministe. Le serveur mitre-mcp (Montimage) expose l’intégralité d’ATT&CK sous forme d’outils interrogeables, avec des recherches en temps constant grâce à des index préconstruits [32]. Lorsqu’il a besoin de la description d’une technique, le modèle ne récupère pas un voisin plausible : il exécute une requête structurée et obtient la définition exacte, de façon reproductible et journalisable. La surface d’empoisonnement par similarité disparaît ; elle se déplace vers la chaîne d’approvisionnement du serveur lui-même, objet de l’axe 4.

Une réserve sur le référentiel. Une analyse conjointe d’Anthropic et de Verizon, qui a cartographié 832 comptes malveillants et 13 873 actions sur la version 18 d’ATT&CK (482 sous-techniques, 14 tactiques), souligne qu’ATT&CK ne capture pas encore l’orchestration autonome par des agents d’IA, marque des campagnes les plus avancées [33]. Le référentiel ancré reste donc une borne inférieure de la menace.

Ce que la littérature empirique mesure sur le MCP

La littérature savante sur le MCP a moins de deux ans, mais quatre travaux portent directement sur la question de l’essai : combien d’outils exposer, ce que l’ancrage fait gagner, ce qu’il tient en conditions réelles, et ce qu’il coûte en sécurité. Le tableau 1.2 les rassemble.

Trop d’outils dégrade la sélection. Gan et Sun mesurent l’effet du nombre d’outils exposés à un modèle [34]. Leur essai de charge présente au modèle N descriptions de serveurs — une seule pertinente, les autres tirées d’un annuaire de plus de 4400 serveurs publics — et fait varier N de 1 à plus de 11 000 sur vingt tâches de recherche. En deçà d’une trentaine de candidats, le bon outil est choisi dans plus de 90 % des cas ; entre trente et soixante-dix, les échecs apparaissent par grappes, les descriptions voisines se confondant ; au-delà d’une centaine, l’échec domine. Deux causes se cumulent : l’*engorgement de l’invite*, le contexte étant saturé de distracteurs, et la *charge de décision*, des outils aux fonctions proches devenant indiscernables. La mesure chiffrée est nette : présenter tous les outils d’un coup donne 13,62 % de sélections correctes pour 2134 jetons d’invite, un pré-filtrage par mots-clés 18,20 % pour 1646 jetons, et une pré-sélection sémantique n’injectant que le meilleur

candidat 43,13 % pour 1084 jetons — soit plus du triple de justesse pour la moitié des jetons. **L'appel à la demande restaure l'efficacité.** Fei, Zheng et Feng inversent la logique : plutôt que d'injecter tous les schémas dès le départ, le modèle énonce lui-même le besoin qu'il ne peut satisfaire, et un routage à deux étages — d'abord le serveur, ensuite l'outil — ne lui renvoie que ce qu'il a demandé [35]. Sur un corpus de 308 serveurs et 2797 outils issus du dépôt officiel, le bon outil est retrouvé parmi près de trois mille candidats représentant 248 100 jetons, et la consommation chute de 98 % sur APIBank à justesse maintenue. L'enseignement pour l'essai est direct : le coût de l'ancrage est une variable de conception, non une fatalité du protocole.

En conditions réelles, l'apport reste partiel. MCP-Universe (Salesforce AI Research) évalue les modèles sur 231 tâches réparties en six domaines et servies par 11 serveurs MCP réels, avec des évaluateurs qui vérifient l'exécution plutôt que la forme de la réponse [36]. Aucun modèle ne franchit la moitié des tâches : GPT-5 réussit 43,72 %, Grok-4 33,33 % et Claude-4.0-Sonnet 29,44 %. Deux causes d'échec reviennent, et toutes deux concernent l'essai : le contexte enfle avec le nombre d'étapes, et les outils inconnus sont mal exploités. La tâche de l'essai — un référentiel unique, en lecture seule, sur un nombre d'étapes borné — se situe précisément dans le cas favorable que ce banc d'essai isole par contraste.

La sécurité du protocole reste en retard. Yang, Wu et Chen proposent la première formalisation systématique des surfaces d'attaque du MCP : 17 types d'attaques répartis sur quatre surfaces — interaction avec l'utilisateur, client, transport et serveur — éprouvés sur trois hôtes du commerce [37]. Plus de 85 % de ces attaques compromettent au moins une plateforme, les failles de protocole et d'implémentation atteignant 100 % de réussite, et les protections intégrées n'en arrêtent qu'une fraction. Le constat prolonge l'axe 4 : ce qui est standardisé, c'est l'accès, pas sa sûreté.

Travail	Dispositif	Résultat clé
Gan et Sun (2025) [34]	Essai de charge, N de 1 à 11 000 serveurs, annuaire de 4400+	Effondrement au-delà de 100 outils ; 13,62 % → 43,13 % avec pré-sélection, jetons divisés par deux
MCP-Zero (2025) [35]	308 serveurs, 2797 outils ; appel d'outil à la demande	Bon outil parmi ≈ 3000 candidats (248 100 jetons) ; −98 % de jetons sur APIBank
MCP-Universe (2025) [36]	231 tâches, 11 serveurs réels, 6 domaines, évaluation par exécution	GPT-5 43,72 %, Grok-4 33,33 %, Claude-4.0-Sonnet 29,44 % ; contexte long et outils inconnus
MCPSecBench (2025) [37]	17 attaques, 4 surfaces, 3 hôtes du commerce	>85 % des attaques réussissent sur au moins un hôte ; protocole et implémentation à 100 %

TABLEAU 1.2 – Ce que la littérature empirique établit sur le MCP (axe 3).

Ce que ces quatre travaux imposent à la conception du serveur. Ils convergent vers une règle simple : un serveur MCP utile est un serveur *étroit*. Exposer peu d’outils, aux périmètres nettement séparés, préserve la justesse de sélection que le premier travail voit s’effondrer ; ne servir que la réponse demandée, sans contexte superflu, préserve le budget de jetons que le deuxième optimise ; borner le nombre d’étapes évite la dérive que le troisième observe ; et n’offrir aucune écriture réduit la surface que le quatrième mesure. Ces quatre contraintes sont celles retenues pour le serveur de l’essai.

Conclusion : L’accès déterministe est l’aboutissement de l’arc de l’ancrage, et le MCP en est désormais le standard sous gouvernance neutre. La littérature empirique en fixe déjà les conditions d’emploi — peu d’outils, servis à la demande, sur un nombre d’étapes borné — mais elle mesure l’ancrage outillé sur des tâches génériques, jamais sur le *mapping* ATT&CK en conditions de SOC : c’est l’un des buts directs de l’essai (QR3).

1.1.4 Axe 4 — Le revers : sécurité des agents outillés et du MCP

Les trois premiers axes construisent l’argument pour l’ancrage outillé ; le quatrième en examine le revers. La bascule est fondamentale : un LLM isolé qui se trompe produit un texte erroné ; un LLM connecté à des outils qui se trompe peut exécuter une action sur un système réel. Le risque passe du registre de l’erreur à celui du dommage, ce que l’OWASP nomme *excessive agency* dans son classement 2025 des risques des applications LLM [38].

La faille fondatrice : l’injection indirecte de prompt

On distingue l’injection directe (l’attaquant tape l’instruction) de l’injection indirecte, plus insidieuse : l’instruction est cachée dans un contenu tiers — page web, courriel, résultat d’outil — que le modèle lit au cours de sa tâche. Greshake *et al.* (AISec, ACM CCS 2023) en ont établi le principe et l’ont démontré sur des applications réelles : parce que le modèle ne distingue pas les données qu’il traite des instructions qu’il doit suivre, un contenu piégé le détourne à distance [39]. Deux bancs d’essai quantifient la vulnérabilité des agents outillés, cas exact du MCP. InjecAgent (ACL 2024) compose 1054 cas à partir de 17 outils légitimes et 62 outils d’attaquant : un agent GPT-4 guidé par ReAct est détourné dans 24 % des cas, 47 % avec des invites d’attaque renforcées [40]. AgentDojo (NeurIPS 2024) va plus loin avec un environnement dynamique de 97 tâches réalistes et 629 cas de sécurité qui mesure conjointement utilité et résistance : sous attaque, l’utilité de GPT-4o chute d’environ 69 % à 50 %, avec des taux de réussite d’attaque atteignant 53 % [41].

Conclusion : Les agents outillés sont, en l'état, largement vulnérables, et la vulnérabilité tient à une propriété du modèle — il suit ce qu'il lit — que ni la taille ni l'alignement n'ont résolue (QR4).

Ce que le MCP ajoute — mesuré sur des serveurs réels

Taxonomie. L'étude de paysage de Hou *et al.* fait référence : elle organise les menaces le long du cycle de vie du serveur — création (collision de noms, installateur usurpé, code injecté), opération (conflits de noms d'outils, chevauchement de commandes, évocation du bac à sable) et mise à jour (persistance de privilèges, redéploiement de versions vulnérables, dérive de configuration) — et identifie un mécanisme central, la propagation de confiance : dans un hôte combinant plusieurs serveurs, un seul serveur compromis peut influencer le comportement des outils légitimes [1]. La figure 1.4 en donne une lecture opérationnelle.

L'empoisonnement d'outil (*tool poisoning*) est la menace spécifique qui dicte la conception à venir : les instructions malveillantes sont glissées non dans les données échangées mais dans la *description* de l'outil, ce texte que le modèle lit pour savoir quand et comment l'employer. Révélée par Invariant Labs en avril 2025 avec une démonstration d'exfiltration de conversations [42], elle a été mesurée à grande échelle par MCPTox (AAAI 2026) : à partir de 45 serveurs réels et de 353 outils authentiques, plus de 1300 cas d'attaque évalués sur 20 modèles donnent un taux de réussite moyen de 36,5 %, culminant à 72,8 % (o1-mini), le taux de refus restant partout inférieur à 3 % [2]. Fait marquant, en écho aux axes 1 et 3, les modèles les plus capables sont souvent les plus vulnérables, l'attaque exploitant leur meilleure aptitude à suivre les instructions.

L'état de l'écosystème. La première étude empirique à grande échelle, publiée dans ACM TOSEM, a analysé 1899 serveurs MCP libres en combinant analyse statique générale et scanner spécifique [43] : huit classes de vulnérabilités dont trois seulement recoupent le logiciel classique ; 7,2 % des serveurs portent une vulnérabilité générale et 5,5 % un empoisonnement d'outil ; 66 % présentent des odeurs de code et 14,4 % des motifs de bogues connus. Côté vulnérabilités avérées, CVE-2025-6514 (CVSS 9,6) permettait l'exécution de commandes arbitraires via l'utilitaire mcp-remote [3], et CVE-2025-49596 (CVSS 9,4) exploitait l'absence d'authentification par défaut de l'inspecteur MCP [4] ; un audit outillé (MCPSafetyScanner) avait dès avril 2025 obtenu exécution de code et vol d'identifiants à travers des serveurs courants [44].

Conclusion : Ces failles ne sont pas exotiques : la plupart sont des manquements aux principes classiques (validation des entrées, moindre privilège, authentification). Une

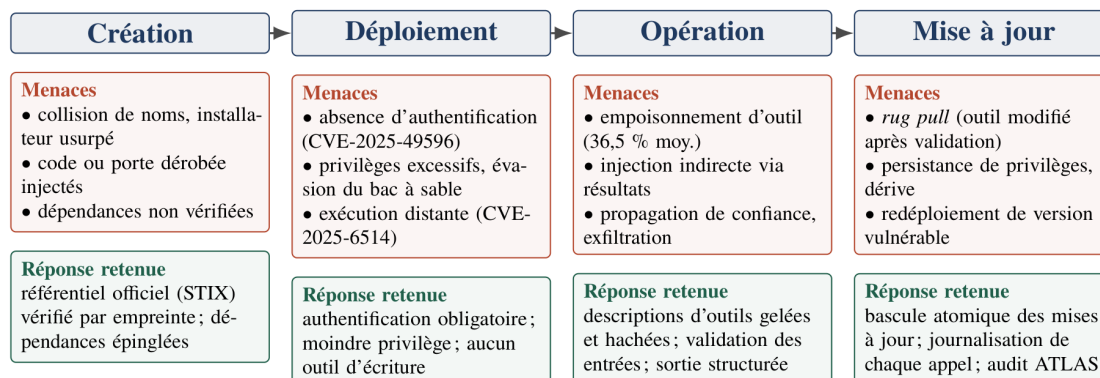


FIGURE 1.4 – Surface d’attaque d’un serveur MCP le long de son cycle de vie (taxonomie adaptée de Hou et al. [1], chiffres de MCPTox [2] et des CVE [3, 4]) et réponses de conception retenues pour le serveur de l’essai.

seule classe est nouvelle — la description d’un outil devient un vecteur d’instruction — et c’est elle qui impose des contrôles spécifiques au MCP (QR4).

Un modèle de sécurité en rattrapage, et les réponses émergentes

La spécification. Une part des garde-fous de consentement et de bonnes pratiques y est formulée en *should* (recommandé) plutôt qu’en *must* (obligatoire), ce qui les rend optionnels pour l’implémenteur; la spécification prévient d’ailleurs elle-même que les descriptions et annotations d’outils doivent être considérées comme non fiables sauf serveur de confiance [6]. Les révisions récentes comportent de véritables *must*, surtout sur l’autorisation (tableau 1.1), mais rien n’oblige un serveur à être en lecture seule, à journaliser, ni à figer ses descriptions : ces propriétés relèvent de la conception de chaque serveur.

Les organismes. Le centre de sécurité de l’IA de la NSA constate, dans une fiche de mai 2026, que la prolifération du MCP a devancé le développement de son modèle de sécurité² [45]. L’OWASP a publié un classement des risques des applications LLM (2025) puis des applications agentiques (2026) [38]. Formaliser ces menaces dans un modèle auditable suppose une taxonomie dédiée à l’IA : MITRE ATLAS, calqué sur ATT&CK mais orienté vers les attaques contre les systèmes d’IA, couvre désormais explicitement la compromission de serveurs MCP et l’injection indirecte [46]. Au Canada, cette exigence d’auditabilité devient légale : la *Loi sur la protection des cybersystèmes essentiels* (projet de loi C-8, sanction royale en juin 2026, qui reprend le projet C-26 devenu caduc) impose aux exploitants désignés d’infrastructures critiques un programme de cybersécurité documenté et la déclaration

2. La co-signature de la CISA n’apparaît pas dans le communiqué officiel; la référence est attribuée à la seule NSA (AISC).

des incidents [47].

La défense outillée. Le tableau 1.3 rassemble les mesures ; côté défense active, l’automatisation de l’audit progresse. PentestGPT (USENIX Security 2024) coordonne trois modules — raisonnement, génération de commandes, analyse des sorties — autour d’un arbre de tâches qui maintient le fil de l’investigation, et augmente le taux de complétion des épreuves de 228,6 % par rapport à GPT-3.5, sous supervision humaine [48]. Des scanners spécifiques (mcp-scan, MCPSafetyScanner) détectent l’empoisonnement de descriptions [42, 44], et les passerelles de médiation qui filtrent et journalisent les appels sont la piste d’architecture la plus citée [1].

Travail (lieu)	Dispositif	Résultat clé
PoisonedRAG (USENIX Sec’25) [28]	5 textes injectés dans 2,6 M de documents	90 % de réussite, jusqu’à 97 %
InjecAgent (ACL’24) [40]	1054 cas, 17 outils légitimes, 62 d’attaquant	GPT-4 + ReAct détourné à 24 %, 47 % renforcé
AgentDojo (NeurIPS’24) [41]	97 tâches, 629 cas de sécurité, utilité + attaque	GPT-4o : utilité 69 % → 50 % ; attaque $\leq$ 53 %
MCPTox (AAAI’26) [2]	45 serveurs réels, 353 outils, >1300 cas, 20 modèles	Moy. 36,5 %, max 72,8 % ; refus < 3 %
Hasan <i>et al.</i> (TOSEM’26) [43]	1899 serveurs libres, analyse statique + scanner	7,2 % vulnérables, 5,5 % empoisonnés, 66 % d’odeurs de code
CVE-2025-6514 / 49596 [3, 4]	mcp-remote ; inspecteur MCP	CVSS 9,6 et 9,4 : exécution de code à distance

TABEAU 1.3 – Mesures de la vulnérabilité des agents outillés et du MCP (axe 4).

Le rug pull : pourquoi la confiance ne doit jamais être acquise

Un serveur se comporte correctement au moment de la validation initiale, puis modifie silencieusement ses outils une fois la confiance accordée [1]. Appliqué à notre cas, un outil de *mapping* pourrait retirer discrètement certaines techniques de ses résultats, rendant des pans entiers d’attaques invisibles sans que l’analyste ne perçoive la dérive. C’est ce scénario, plus que l’exécution de code, qui dicte les choix du serveur développé dans l’essai : aucun outil d’écriture, référentiel en lecture seule vérifié par empreinte, descriptions d’outils gelées, bascule atomique des mises à jour et journalisation systé-

matique de chaque appel (figure 1.4).

Conclusion : La voie d'un audit de sécurité outillé et ancré est ouverte : c'est ce que le chapitre suivant prolonge avec un flux multi-MCP fondé sur ATLAS, sous contrainte réglementaire canadienne (**QR4**).

1.1.5 Synthèse : acquis et lacunes

Le tableau 1.4 met en regard, pour chaque question de revue, l'acquis chiffré de la littérature et la lacune qu'elle laisse ouverte.

	Acquis (chiffré)	Lacune actuelle
QR1	Les LLM aident mais hallucinent ; leur mesure est piégée par la contamination (F1 68 % → 3 % [20]) ; le meilleur modèle généraliste plafonne à 0,64 F1 en <i>mapping</i> [17] ; l'OT est un angle mort [13, 23]	Aucune évaluation sur des journaux OT réels ; les bancs d'essai portent sur du texte de renseignement en anglais, et la contamination interdit de les réutiliser tels quels
QR2	Le RAG ancre et cite, mais un voisin n'est pas une définition, le résultat n'est pas reproductible, et cinq textes suffisent à 90 % de réussite d'attaque [28]	Pas de comparaison contrôlée RAG vs accès déterministe sur le <i>mapping</i> ATT&CK, à modèles et journaux identiques
QR3	Un modèle outillé de 6,7 G bat GPT-3 175 G [29] ; le MCP standardise l'accès déterministe sous gouvernance neutre [5, 12] ; au-delà d'une centaine d'outils exposés la sélection s'effondre (13,6 % contre 43,1 % avec pré-sélection [34]), et l'appel à la demande économise 98 % des jetons [35]	Les bancs d'essai MCP portent sur des tâches génériques [36] ; la supériorité théorique de l'accès déterministe (justesse, reproductibilité, latence) n'est pas mesurée sur le <i>mapping</i> ATT&CK en conditions de SOC

	Acquis (chiffré)	Lacune actuelle
QR4	Le MCP déplace le risque du texte à l'action ; les serveurs réels sont vulnérables (jusqu'à 72,8 % de réussite [2], 5,5 % empoisonnés [43]) ; la spécification laisse la lecture seule et la journalisation à l'implémenteur	Le « serveur MCP sûr par conception pour un SOC régulé » et son audit automatisé ancré dans ATLAS restent peu traités, a fortiori sous le cadre de la LPCE

TABEAU 1.4 – Ce que la revue établit et ce qu'elle laisse ouvert, par question de revue.

Trois fils se dégagent. **D'abord**, un motif transversal : à chaque axe, les modèles les plus capables sont les plus exposés — au code non sûr (CyberSecEval), à l'excès de confiance mémorisée (*inverse scaling*), à l'empoisonnement d'outil (MCPTox) — parce que l'attaque exploite l'aptitude même qui fait leur valeur, suivre finement une instruction. La réponse n'est donc pas un meilleur modèle mais un meilleur *cadre* : ancrer, borner, journaliser. **Ensuite**, l'arc de l'ancrage ne supprime pas la surface d'attaque, il la déplace — du modèle vers la base (RAG), puis de la base vers le serveur (MCP) — et chaque déplacement rend la surface plus petite et plus auditable, à condition que le serveur soit conçu pour l'être. **Enfin**, la littérature savante sur le MCP a moins de deux ans : elle mesure abondamment sa vulnérabilité mais très peu son apport en conditions opérationnelles. C'est dans cet écart que se situe l'essai ; la manière dont il répond à chacune de ces lacunes est exposée dans sa conclusion, à la lumière des travaux réalisés.

1.2 MÉTHODOLOGIE D'ÉVALUATION

Le projet intègre le référentiel MITRE ATT&CK dans un écosystème MCP. La mission confiée au modèle est de produire des moyens de mitigation, et la qualité d'une mitigation dépend directement de la justesse du mapping qui la précède. On évalue donc les modèles sur cette tâche, aux trois niveaux du référentiel, dans deux environnements : les technologies de l'information (TI) et les technologies opérationnelles (TO).

Les travaux les plus proches — RAM, CTIBench — ont été publiés entre 2024 et le début de 2026 et portent sur des modèles aujourd'hui dépassés : GPT-4-Turbo, Llama 3, Mistral 7B, IBM Granite. Ils traitent par ailleurs de tâches voisines mais distinctes, comme le mapping de vulnérabilités CVE, la génération de règles SIEM ou la conversion de rapports de renseignement. Aucun n'évalue la matrice ATT&CK for ICS, ni la version v19.2 du référentiel. C'est cette absence d'évaluation actualisée et alignée sur notre cas d'usage qui motive la campagne.

1.2.1 Construction du corpus

1.2.1.1 Pourquoi générer les journaux

Les journaux sont produits en laboratoire plutôt que repris d'un jeu public. La raison est la contamination : si un jeu de données est public, le modèle a pu voir pendant son entraînement les journaux *et* leurs identifiants MITRE. Il répondrait alors de mémoire et non par raisonnement, et la campagne mesurerait sa mémorisation. En générant nous-mêmes les journaux, on garantit que leur forme exacte n'a jamais été indexée.

L'évaluation porte donc sur la capacité du modèle à mapper correctement des activités malveillantes **connues**, et non sur sa capacité à distinguer le légitime du malveillant.

1.2.1.2 Génération

Le laboratoire virtuel couvre deux zones. La zone TI comprend un contrôleur de domaine Windows Server DC01 instrumenté avec Sysmon, et un serveur Linux web01 instrumenté avec auditd. La zone TO est un réseau ICS composé d'une simulation de procédé GRFICS v3 et d'une sonde réseau Zeek.

Des attaques unitaires ciblent chaque zone via un harnais qui produit les traces. Les résultats sont capturés dans les journaux Sysmon et Security pour DC01, dans audit.log pour web01, dans les journaux Zeek (Modbus) et le journal d'audit de l'hôte pour la zone TO. Un script découpe ensuite ces journaux en échantillons et construit la vérité de terrain avec des identifiants validés contre la version courante du référentiel.

1.2.1.3 Un triplet par échantillon

Chaque échantillon correspond à une seule action offensive et porte exactement un triplet attendu : une tactique, une technique, et une sous-technique lorsque la technique en possède une, la mention « aucune » sinon.

Une technique peut être rattachée à plusieurs tactiques dans le référentiel — T1053 relève à la fois de Execution, Persistence et Privilege Escalation. Dans ce cas, on retient la seule tactique correspondant à l'intention réelle du scénario exécuté. L'attaque ayant été lancée par nous, cette intention est connue sans ambiguïté.

1.2.1.4 Composition du corpus

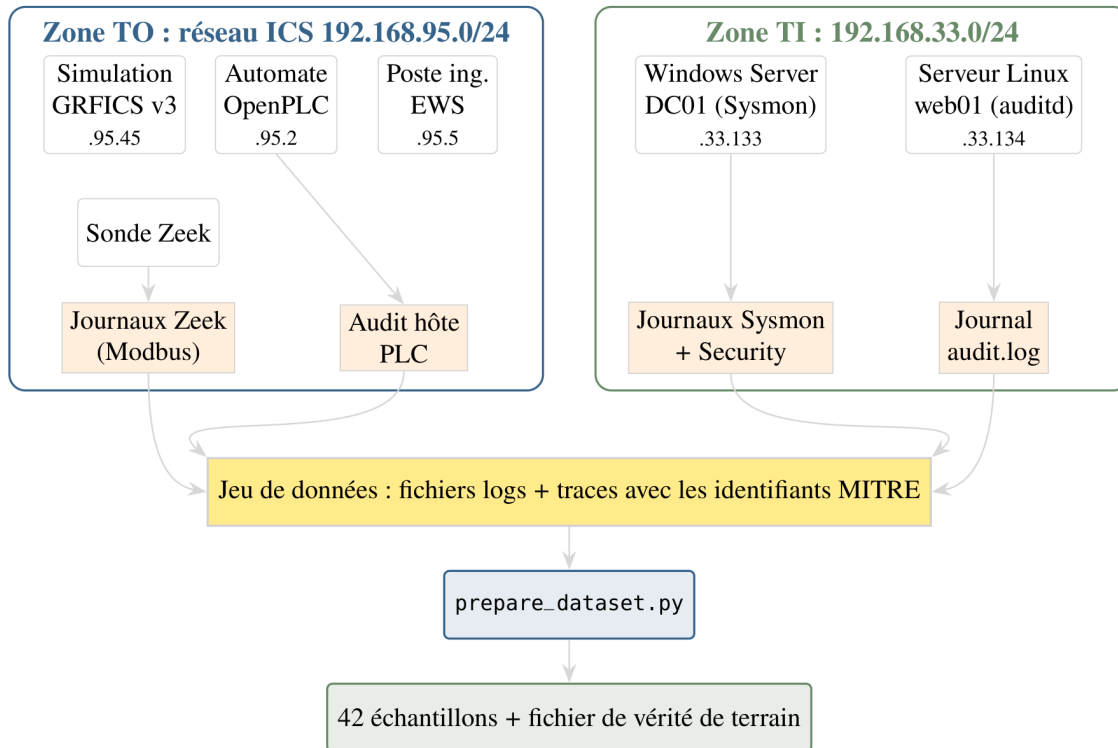


FIGURE 1.5 – Génération de la télémétrie TO et TI.

Zone	Source	Matrice	Nombre	Avec sous-tech.	Techniques couvertes
TI	DC01 — Sysmon + Security	Enterprise	12	7	11
TI	web01 — audit.log	Enterprise	16	14	14
TO	Sonde Zeek — Modbus	ICS	11	1	11
TO	Hôte PLC — audit	ICS	3	0	3
Total			42	22	36

TABLEAU 1.5 – Composition du corpus de la vérité de terrain.

La vérité de terrain complète est reproduite ci-dessous, échantillon par échantillon. Elle fixe le triplet attendu à chaque niveau et sert de référence à tous les calculs qui suivent.

Éch.	Mat.	Tactique	Technique	Sous-technique
S000	ICS	TA0102 — Discovery	T0846 — Remote System Discovery	aucune
S001	ICS	TA0102 — Discovery	T0888 — Remote System Information Discovery	aucune
S002	ICS	TA0100 — Collection	T0861 — Point & Tag Identification	aucune
S003	ICS	TA0100 — Collection	T0801 — Monitor Process State	aucune
S004	ICS	TA0100 — Collection	T0802 — Automated Collection	aucune
S005	ICS	TA0100 — Collection	T0868 — Detect Operating Mode	aucune
S006	ICS	TA0106 — Impair Process Control	T0836 — Modify Parameter	aucune
S007	ICS	TA0106 — Impair Process Control	T1692 — Unauthorized Message	T1692.001 — Command Message
S008	ICS	TA0106 — Impair Process Control	T0806 — Brute Force I/O	aucune
S009	ICS	TA0107 — Inhibit Response Function	T0835 — Manipulate I/O Image	aucune
S010	ICS	TA0107 — Inhibit Response Function	T0814 — Denial of Service	aucune
S011	ICS	TA0104 — Execution	T0807 — Command-Line Interface	aucune
S012	ICS	TA0104 — Execution	T0853 — Scripting	aucune
S013	ICS	TA0110 — Persistence	T0889 — Modify Program	aucune
S014	Ent.	TA0002 — Execution	T1059 — Command and Scripting Interpreter	T1059.004 — Unix Shell

Éch.	Mat.	Tactique	Technique	Sous-technique
S015	Ent.	TA0002 — Execution	T1059 — Command and Scripting Interpreter	T1059.006 — Python
S016	Ent.	TA0004 — Privilege Escalation	T1548 — Abuse Elevation Control Mechanism	T1548.003 — Sudo and Sudo Caching
S017	Ent.	TA0004 — Privilege Escalation	T1548 — Abuse Elevation Control Mechanism	T1548.001 — Setuid and Setgid
S018	Ent.	TA0003 — Persistence	T1053 — Scheduled Task/Job	T1053.003 — Cron
S019	Ent.	TA0003 — Persistence	T1543 — Create or Modify System Process	T1543.002 — Systemd Service
S020	Ent.	TA0003 — Persistence	T1546 — Event Triggered Execution	T1546.004 — Unix Shell Configuration Modification
S021	Ent.	TA0003 — Persistence	T1098 — Account Manipulation	T1098.004 — SSH Authorized Keys
S022	Ent.	TA0003 — Persistence	T1037 — Boot or Logon Initialization Scripts	T1037.004 — RC Scripts
S023	Ent.	TA0006 — Credential Access	T1003 — OS Credential Dumping	T1003.008 — /etc/passwd and /etc/shadow
S024	Ent.	TA0006 — Credential Access	T1552 — Unsecured Credentials	T1552.003 — Shell History
S025	Ent.	TA0005 — Stealth	T1070 — Indicator Removal	T1070.003 — Clear Command History
S026	Ent.	TA0005 — Stealth	T1140 — Deobfuscate/Decode Files or Information	aucune
S027	Ent.	TA0112 — Defense Impairment	T1685 — Disable or Modify Tools	T1685.006 — Clear Linux or Mac System Logs

Éch.	Mat.	Tactique	Technique	Sous-technique
S028	Ent.	TA0112 — Defense Impairment	T1222 — File and Directory Permissions Modification	T1222.002 — Linux and Mac Permissions
S029	Ent.	TA0112 — Defense Impairment	T1686 — Disable or Modify System Firewall	aucune
S030	Ent.	TA0007 — Discovery	T1033 — System Owner/User Discovery	aucune
S031	Ent.	TA0007 — Discovery	T1082 — System Information Discovery	aucune
S032	Ent.	TA0007 — Discovery	T1057 — Process Discovery	aucune
S033	Ent.	TA0007 — Discovery	T1016 — System Network Configuration Discovery	aucune
S034	Ent.	TA0007 — Discovery	T1087 — Account Discovery	T1087.001 — Local Account
S035	Ent.	TA0002 — Execution	T1059 — Command and Scripting Interpreter	T1059.001 — PowerShell
S036	Ent.	TA0002 — Execution	T1059 — Command and Scripting Interpreter	T1059.003 — Windows Command Shell
S037	Ent.	TA0005 — Stealth	T1027 — Obfuscated Files or Information	T1027.010 — Command Obfuscation
S038	Ent.	TA0003 — Persistence	T1547 — Boot or Logon Autostart Execution	T1547.001 — Registry Run Keys / Startup Folder
S039	Ent.	TA0003 — Persistence	T1053 — Scheduled Task/Job	T1053.005 — Scheduled Task
S040	Ent.	TA0003 — Persistence	T1543 — Create or Modify System Process	T1543.003 — Windows Service

Éch.	Mat.	Tactique	Technique	Sous-technique
S041	Ent.	TA0112 — Defense Impairment	T1112 — Modify Registry	aucune

TABLEAU 1.6 – Vérité de terrain par échantillon : tactique, technique et sous-technique attendues.

1.2.1.5 Échantillon et exécution : deux unités de compte

Chaque échantillon est soumis trois fois au même modèle. Un modèle de langue ne rend pas nécessairement la même réponse deux fois de suite : répéter l'appel permet de mesurer cette reproductibilité au lieu de la supposer.

Un appel au modèle — un échantillon envoyé, une réponse reçue — est appelé une **exécution**. La campagne compte donc **42 échantillons et 126 exécutions** par modèle, et le fichier de résultats contient une ligne par exécution, non par échantillon.

Les deux unités ne servent pas aux mêmes mesures. Les taux de diagnostic se comptent en **exécutions** : un comportement qui se répète sur les trois exécutions d'un même échantillon s'est produit trois fois, et doit compter trois fois. La stabilité se compte en **échantillons**, puisqu'elle compare les trois exécutions entre elles — il n'y a rien à compter par exécution lorsque la question porte sur leur ensemble.

Un exemple. Si un modèle s'arrête à la tactique sur deux des trois exécutions d'un même échantillon, le compteur de sous-spécification retient 2, et non 1 : le comportement s'est produit deux fois sur les 126 occasions qu'il avait de se produire. Rapporter ce 2 aux 42 échantillons donnerait un taux trois fois trop élevé.

1.2.2 Étiquetage des prédictions

1.2.2.1 Les trois niveaux du référentiel

Le cadre MITRE ATT&CK est hiérarchique : chaque technique appartient à une tactique, et certaines techniques se déclinent en sous-techniques. La figure 1.6 l'illustre avec T1059 et ses variantes.

Une erreur n'a pas la même gravité selon le niveau où elle se produit : confondre deux sous-techniques d'une même technique est une erreur fine, se tromper de tactique est une erreur grossière. La vérité de terrain est donc préparée aux trois niveaux, et chaque niveau est évalué séparément.

Le corpus compte 42 échantillons. Chacun porte un identifiant attendu par niveau. Le modèle rend un identifiant par niveau. On veut savoir à quel point il réussit.

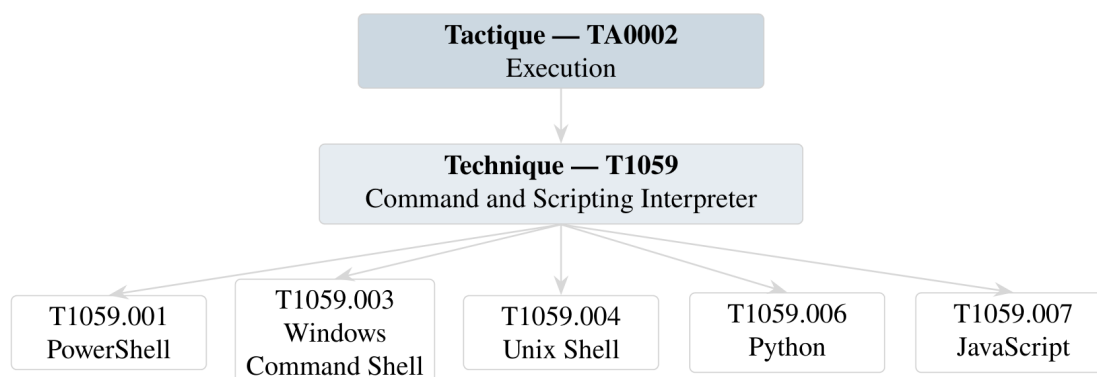


FIGURE 1.6 – Hiérarchie à trois niveaux du référentiel, illustrée par la technique T1059.

Il n’y a pas une seule façon d’échouer. Le modèle peut donner un mauvais identifiant qui existe. Il peut en inventer un qui n’existe pas. Il peut donner un identifiant révoqué dont le successeur est exactement la bonne réponse. Il peut ne rien donner. Ces quatre échecs ne disent pas la même chose sur le modèle, et un score unique les écraserait.

D’où le principe : **on classe chaque prédiction dans une catégorie, avant tout calcul**. Les trois niveaux sont étiquetés et mesurés séparément, car les erreurs n’y ont pas la même gravité : confondre deux sous-techniques d’une même technique est une erreur fine, se tromper de tactique est une erreur grossière.

1.2.2.2 Les cinq catégories

- **VP** — l’identifiant attendu est prédit.
- **FP** — un identifiant qui existe, mais pas le bon. Attendu T1059.001, prédit T1547.001.
- **H** (hallucination) — un identifiant qui n’existe nulle part. T1059.999. Un identifiant *déprécié* — retiré du référentiel sans remplaçant officiel — n’en est pas une : il existe. Il est compté FP et suivi par un compteur dédié ; faute de remplaçant officiel à vérifier, il ne peut pas être crédité O.
- **O** (obsolète) — un identifiant révoqué dont le remplaçant officiel est celui attendu. Prédit T1086, attendu T1059.001. Le raisonnement est juste, le référentiel du modèle date. C’est la seule catégorie dont le traitement change d’une base de calcul à l’autre. Elle n’est détectable qu’aux niveaux technique et sous-technique ; au niveau tactique, le cas est indiscernable et reste compté FP.
- **A** (abstention) — rien n’est prédit, le champ est vide. Une réponse inexploitable imputable au modèle — format non respecté, JSON invalide, réponse vide — devient une abstention aux trois niveaux, avec un compteur dédié ; les F1 sont aussi recalculés

hors rejets.

Deux propriétés comptent. **Disjointes** : une prédiction reçoit une catégorie, une seule. **Exhaustives** : toute prédiction en reçoit une. Il en découle, sur les N échantillons évalués à un niveau, l'égalité $N = VP + FP + H + O + A$. C'est l'outil de contrôle : si elle ne tombe pas juste, il y a une erreur de comptage.

Ce qu'on perdrait sans cette séparation. En comptant seulement des bonnes et des mauvaises réponses, trois informations disparaîtraient. Le taux d'hallucination, H étant noyé dans les mauvaises — or un modèle qui invente des identifiants est disqualifié pour un usage opérationnel, quel que soit son score. La base tolérante, O étant noyé de la même façon — or la résolution des identifiants révoqués est précisément ce que la campagne veut mesurer. Et la distinction entre A et l'erreur : l'abstention renvoie le cas à l'analyste, l'erreur l'oriente vers une cible fausse.

Pourquoi FN ne figure pas dans la liste. Les cinq catégories décrivent ce que le modèle a écrit, et la liste est complète. Le faux négatif désigne autre chose : la réponse attendue n'a pas été retrouvée. Cela se produit dans quatre des cinq catégories — FP , H , A , et O en base stricte. Ce n'est donc pas une catégorie de plus mais un regroupement qui traverse les autres ; l'ajouter romprait l'exclusivité mutuelle et l'égalité ci-dessus. Les catégories disent ce que le modèle a écrit, les regroupements du tableau 1.8 disent ce que ça vaut.

1.2.2.3 Pourquoi le vrai négatif n'est pas comptabilisé

La version figée du référentiel (v19.2) compte 794 techniques. Pour chaque échantillon, le modèle en écarte implicitement 793. En les comptant, l'exactitude atteindrait 99,9 % pour tous les modèles, y compris pour un modèle qui se tromperait systématiquement : l'indicateur cesserait de discriminer.

C'est pourquoi la précision et le rappel sont les métriques adaptées ici : **ni l'une ni l'autre n'utilise le vrai négatif**. Elles sont construites pour les cas où les négatifs écrasent numériquement les positifs.

1.2.2.4 Le niveau sous-technique

Les niveaux tactique et technique sont évalués sur les 42 échantillons. Le niveau sous-technique est le seul où le corpus se dédouble : **22 échantillons** possèdent une sous-technique attendue, **20 n'en possèdent aucune** — leur technique est terminale. Le $F1$ sous-technique est calculé sur les 22, puisque la question posée est : lorsqu'une sous-technique existe, le modèle la retrouve-t-il ? Les 20 autres ne peuvent pas y contribuer, il n'y a rien à retrouver.

Sur les 42, un modèle répondant « aucune » partout sans analyse tomberait juste sur les 20 terminaux et obtiendrait un $F1$ de 0,476 — la moitié du score maximal, sans aucune capacité.

Ces cas sont triviaux, connus d'avance dès que la technique l'est ; les inclure comprimerait l'échelle vers le haut. Sur 22, ce modèle obtient zéro. La capacité à reconnaître une technique terminale reste mesurée, par le **taux de sur-spécification**, rapporté sur les 20.

« aucune » **n'est pas le vide**. Le critère est la forme de la réponse, pas la vérité attendue. « aucune » est une réponse : le modèle affirme que la technique n'a pas de sous-technique. C'est vérifiable, donc faux lorsqu'une sous-technique existait, et compté FP. Le champ vide est une absence de réponse : rien n'est affirmé, c'est une abstention. Confondre les deux fausse deux indicateurs différents — traiter « aucune » comme une abstention mélange le silence et une prise de position ; traiter le vide comme « aucune » crédite le modèle d'une réponse qu'il n'a pas donnée. La consigne impose donc un champ sous_technique toujours présent : un identifiant, la chaîne « aucune », ou une chaîne vide.

Attendu	Prédit	Catégorie	Lecture
Les 22 échantillons évalués			
T1059.001	T1059.001	VP	Réponse exacte.
T1059.001	T1059.003	FP	Confusion entre variantes.
T1059.001	« aucune »	FP	Absence affirmée à tort ; compteur dédié.
T1059.001	champ vide	A	Abstention ; alimente la sous-spécification N2.
T1059.001	T1086 (révoqué)	O	Successeur officiel = réponse attendue.
T1059.001	T1059.999	H	Identifiant inexistant.
Les 20 échantillons terminaux — hors du F1			
aucune	« aucune »	—	Comportement attendu.
aucune	champ vide	—	Non compté.
aucune	une sous-technique	—	Sur-spécification ; compteur dédié, sur 20.

TABLEAU 1.7 – Étiquetage du niveau sous-technique.

1.2.2.5 *Attributs de diagnostic*

Ces indicateurs ne sont pas des catégories : ils se superposent à la classification et expliquent *comment* le modèle échoue, là où le F1 mesure *combien* il réussit. Ils se comptent en exécutions (section 1.2.1.5), sauf la stabilité, qui se compte en échantillons.

- **Sous-spécification N1** = exécutions où la tactique est fournie sans technique / exécutions évaluées. Le modèle a compris l'intention, pas le mécanisme.
- **Sous-spécification N2** = exécutions où la technique est correcte et la sous-technique omise / exécutions portant sur un échantillon à sous-technique. Bon mécanisme, granularité insuffisante pour des mitigations précises.
- **Sur-spécification** = exécutions affirmant une sous-technique sur une technique terminale / exécutions portant sur un échantillon terminal. L'inverse du précédent : inventer de la précision.
- **Sous-technique niée à tort** = nombre d'exécutions répondant « aucune » alors qu'une sous-technique existait. Effectif brut, en complément du F1.
- **Mauvaise matrice** = nombre d'identifiants prédits hors de la matrice de l'échantillon, les trois niveaux confondus. Effectif brut.
- **Stabilité** = échantillons dont toutes les exécutions rendent un triplet identique / échantillons évalués au moins deux fois. Mesure la reproductibilité, indépendamment de la justesse.

Cas exclu du calcul. Un échec imputable à l'infrastructure d'appel — quota épuisé, requête annulée, délai dépassé, coupure réseau — ne mesure pas le modèle. Le harnais rejoue l'appel avec attente exponentielle ; toute itération abandonnée malgré les tentatives est marquée (erreur_infrastructure) puis écartée, son effectif étant rapporté par modèle.

1.2.3 Calcul des métriques

1.2.3.1 Deux questions, deux populations

La précision et le rappel ne sont pas deux versions du même chiffre. Ils portent sur deux populations.

- **Précision** — population : les prédictions émises. Question : combien étaient justes ? Dénominateur $VP + FP + H + O$. Les abstentions n'y sont pas, puisque rien n'a été émis.
- **Rappel** — population : les échantillons évalués. Question : sur combien le modèle a-t-il produit la bonne réponse ? Dénominateur N . Les abstentions y sont.

Deux dénominateurs différents, parce que deux questions différentes. Il en découle la **règle d'appariement** : une prédiction fausse compte comme échec au rappel, au même titre qu'une absence de prédiction. Dans les deux cas, l'échantillon n'a pas été qualifié.

Pourquoi cette règle. Un modèle qui répond aux 42 échantillons et se trompe 36 fois a 6 bonnes réponses : son rappel vaut $6/42$, soit 14 %. Si seules les abstentions comptaient comme échec, ce modèle n'en aurait aucune et son rappel vaudrait $6/6$, soit 100 % — un modèle qui échoue 86 % du temps serait crédité d'un rappel parfait. C'est la seule règle qui empêche

d'améliorer son rappel en devinant. Une prédiction fausse pénalise donc la précision, parce que le modèle a affirmé du faux, et le rappel, parce que l'échantillon reste non qualifié : ce ne sont pas deux sanctions pour une même faute, mais deux questions auxquelles la réponse est négative.

1.2.3.2 Les deux bases

Un mapping juste mais daté ne doit être confondu ni avec une réussite pleine ni avec une erreur ordinaire. La **base stricte** prend le référentiel courant pour juge : l'obsolète est une erreur, puisqu'un analyste qui le recevrait chercherait une entrée qui n'existe plus. La **base tolérante** prend le raisonnement pour juge : l'obsolète est crédité, puisque la table de correspondance MITRE atteste que la réponse désignait la bonne réalité. La base stricte fournit les métriques principales ; l'écart entre les deux isole la part des erreurs imputable à la fraîcheur des connaissances.

	Numérateur	Dénom. précision	Dénom. rappel
Base stricte	VP	$VP + FP + H + O$	N
Base tolérante	$VP + O$	$VP + FP + H + O$	N

TABLEAU 1.8 – Ce qui change d'une base à l'autre.

Une seule différence entre les deux lignes : O passe au numérateur. Les dénominateurs sont identiques — le modèle n'a pas émis plus ou moins de prédictions, et le corpus n'a pas changé de taille. C'est cette invariance qui rend l'écart entre les deux bases interprétable : il mesure O , et rien d'autre.

Pourquoi un tableau plutôt qu'une réaffectation. On écrit souvent, pour passer d'une base à l'autre, des affectations du type « FP devient $FP + O$ ». Le symbole y désigne deux choses dans une même ligne, et la formule qui le réutilise ensuite devient ambiguë : selon la lecture, l'obsolète est compté une fois ou deux, et le résultat change. Ici les catégories ne bougent jamais ; seule la façon de les regrouper change, et chaque symbole garde un sens unique dans tout le document.

1.2.3.3 Précision, rappel et $F1$

Précision stricte = $VP / (VP + FP + H + O)$ **Rappel strict** = VP / N

Précision tolérante = $(VP + O) / (VP + FP + H + O)$ **Rappel tolérant** = $(VP + O) / N$

$F1$ = $2 \times \text{Précision} \times \text{Rappel} / (\text{Précision} + \text{Rappel})$, dans chaque base.

Trois égalités permettent de contrôler n'importe quel tableau de résultats sans refaire l'analyse : réponses justes + cibles manquées = N ; cibles manquées = affirmations fausses + A ;

réponses justes + affirmations fausses = nombre d'affirmations émises, identique dans les deux bases.

Une propriété à connaître. Précision et rappel coïncident lorsqu'un modèle se prononce systématiquement et ne produit aucun identifiant obsolète ; ils divergent dès que l'une de ces deux conditions cesse d'être remplie. C'est une conséquence arithmétique du cadre, non un défaut : ce sont l'abstention et le reclassement de *O* qui font diverger les trois indicateurs.

1.2.3.4 Exemple de vérification

Dix échantillons à un niveau donné : $VP = 5$, $FP = 2$, $H = 1$, $O = 1$, $A = 1$, soit 9 affirmations émises et $N = 10$.

	Justes	Faussees	Manquées	Précision	Rappel	F1
Stricte	5	4	5	$5/9 = 0,556$	$5/10 = 0,500$	0,526
Tolérante	6	3	4	$6/9 = 0,667$	$6/10 = 0,600$	0,632

TABLEAU 1.9 – Application numérique dans les deux bases.

Les trois égalités se vérifient : $5 + 5 = 10$ et $6 + 4 = 10$; $5 = 4 + 1$ et $4 = 3 + 1$; 9 affirmations sur les deux lignes. L'écart de F1, 0,632 moins 0,526, mesure ici le coût de la seule prédiction obsolète.

1.2.3.5 Indicateurs dérivés

- **Écart d'obsolescence** = F1 tolérant - F1 strict.
- **Taux d'obsolescence** = O / affirmations émises, les trois niveaux confondus. Le référentiel interne du modèle est-il à jour ?
- **Taux d'hallucination** = H / affirmations émises, les trois niveaux confondus. Le modèle fabrique-t-il des identifiants ?
- **Taux d'abstention** = A / N , rapporté par niveau. Un modèle peut être précis parce qu'il ne se prononce que lorsqu'il est sûr : ce taux complète la lecture de la précision.
- **Latence** = horodatage de fin - horodatage de début de l'appel réussi ; les pauses du harnais et les attentes de reprise n'en font pas partie. Elle produit une valeur par exécution, soit 126 exécutions ($3 \text{ exécutions} \times 42 \text{ échantillons}$). Les latences sont triées par ordre croissant, $v[0]$ étant la plus rapide. On utilise la convention d'interpolation de NumPy pour calculer le p50 et le p95 : le rang cherché vaut $k = (n - 1) \times q$ et, en notant i la partie entière de k , la valeur est $v[i] + (v[i+1] - v[i]) \times (k - i)$.

- **p50 (médiane)** — $q = 0,50$, soit $k = 125 \times 0,50 = 62,5$: la valeur se lit à mi-chemin entre la 63e et la 64e latence. La moitié des exécutions sont plus rapides : c'est le temps de traitement courant du modèle.
- **p95** — $q = 0,95$, soit $k = 125 \times 0,95 = 118,75$: la valeur se lit aux trois quarts entre la 119e et la 120e latence. 95 % des exécutions sont plus rapides : c'est le pire cas ordinaire, celui qui dimensionne un usage en exploitation et qu'une moyenne masquerait.
- **Jetons** = jetons d'entrée + jetons de sortie, jetons de raisonnement compris lorsque le fournisseur les facture. Rapportés en moyenne par exécution.
- **Efficacité** = $1000 \times \text{F1 technique strict} / \text{jetons moyens par exécution}$, soit le F1 obtenu par millier de jetons consommés. Le F1 technique est retenu car les mitigations MITRE se rattachent aux techniques.

1.2.3.6 Ventilation des résultats

Chaque taux est rapporté avec le dénominateur fixé lors de sa définition — sections 1.2.2.4 et 1.2.2.5 — afin qu'aucune proportion ne soit lue sans son effectif. Toutes les métriques sont produites globalement puis ventilées par matrice — Entreprise, 28 échantillons ; ICS, 14 — avec leurs effectifs bruts. Une différence entre deux modèles n'est retenue que si elle est nette et constante sur les trois exécutions.

Ce cadre est la micro-agrégation usuelle en classification multi-classes à étiquette unique. Il est vérifiable par les trois égalités, non ambigu puisque chaque symbole garde un sens unique, et complet puisque les quatre modes d'échec — confusion, fabrication, obsolescence, abstention — restent mesurables séparément.

1.2.4 Sélection des modèles

1.2.4.1 Critères de sélection

Dix modèles sont retenus, issus de six éditeurs. Le choix suit cinq critères, tous liés à ce que la campagne cherche à mesurer.

- **Récence** — les travaux voisins évaluent des modèles dépassés. On retient la génération courante, seule pertinente pour une recommandation opérationnelle.
- **Deux familles** — les modèles propriétaires correspondent à ce qu'un exploitant utilise aujourd'hui par API ; les modèles à poids ouverts à ce qu'il pourrait héberger lui-même, question qui se pose dès que des journaux d'infrastructure critique quittent le périmètre.
- **Couverture des éditeurs et des gammes** — six éditeurs sont représentés, ce qui évite de mesurer une seule lignée d'entraînement. Pour quatre d'entre eux, deux points de

gamme sont évalués : un modèle de tête et un modèle plus léger de la même lignée. Comparer deux modèles d'un même éditeur isole l'effet de la taille à entraînement constant, ce qu'une comparaison entre éditeurs ne permet pas.

- **Capacité de raisonnement** — le mapping n'est pas une reconnaissance de mots-clés mais un raisonnement contextuel. On retient donc des modèles de la classe raisonnement, ce que la revue de littérature justifie.
- **Appel d'outils** — la phase 2 rejoue la campagne avec le serveur MCP. Les dix modèles doivent donc gérer l'appel d'outils, sans quoi la comparaison avec et sans ancrage serait impossible sur le même panel.

1.2.4.2 Les dix modèles retenus

Modèle	Éditeur	Identifiant d'API	Accès
Modèles propriétaires			
Claude Opus 4.8	Anthropic	claude-opus-4-8	API Anthropic
Claude Sonnet 5	Anthropic	claude-sonnet-5	API Anthropic
GPT-5.6 Sol	OpenAI	gpt-5.6-sol	API OpenAI
GPT-5.6 Luna	OpenAI	gpt-5.6-luna	API OpenAI
Gemini 3.1 Pro	Google	google/gemini-3.1-pro-preview	Vertex AI
Gemini 3.8 Flash	Google	google/gemini-3.8-flash	Vertex AI
Modèles à poids ouverts			
Kimi K3	Moonshot AI	kimi-k3	API Moonshot
DeepSeek V4 Pro	DeepSeek	deepseek-v4-pro	API DeepSeek
DeepSeek V4 Flash	DeepSeek	deepseek-v4-flash	API DeepSeek
Qwen3.8	Alibaba	qwen3.8-2.4t-a95b	Model Studio

TABLEAU 1.10 – Modèles évalués, éditeurs et voies d'accès.

Les six modèles propriétaires proviennent de trois éditeurs distincts et représentent l'état de l'art accessible par API au moment de la campagne. Les quatre modèles à poids ouverts sont interrogés par **les API de premier niveau de leurs éditeurs** — Moonshot AI, DeepSeek, Alibaba Cloud — et non par une passerelle d'inférence. Ce choix écarte les écarts de quantification et de réglage de service propres aux passerelles : les dix modèles sont servis dans des conditions homogènes, ce qui est la condition d'une comparaison entre familles.

Les identifiants d'API sont ceux transmis par le harnais et sont relevés dans chaque enregistrement de résultat, avec la date de la campagne. Cette trace est nécessaire : les identifiants

évoluent, et *gemini-3.1-pro-preview* porte la mention preview, donc peut changer ou disparaître. Le préfixe *google/* des deux modèles Gemini n'appartient pas au modèle : il est imposé par l'endpoint compatible OpenAI de Vertex AI. Un accès par clé AI Studio les désignerait sans préfixe.

1.2.4.3 Conditions d'appel

Les conditions sont identiques pour les dix modèles là où les API le permettent : même consigne système, même format de sortie imposé, même plafond de sortie de 32 768 jetons, trois exécutions par échantillon. Le plafond est assez haut pour ne contraindre aucun modèle sur cette tâche ; le champ *fin_raison* enregistré à chaque appel permet de le vérifier après coup — aucun enregistrement ne doit porter la mention de troncature.

La température fait exception. Elle est fixée à 0 lorsque le paramètre est accepté, et laissée à sa valeur par défaut là où les API l'imposent ou l'ignorent : Anthropic et Moonshot n'acceptent plus de température configurable sur leurs modèles à raisonnement, et le schéma OpenAI ne la transmet pas. La valeur effectivement appliquée est enregistrée à chaque appel. La stabilité inter-exécutions se lit donc comme la reproductibilité du modèle dans ses conditions de service réelles, et non comme un test de déterminisme à température nulle.

L'effort de raisonnement est laissé aux valeurs par défaut de chaque fournisseur. Aucune échelle commune n'existe entre éditeurs : forcer un niveau « maximal » partout ne rendrait pas les modèles comparables, cela les déplacerait vers des points arbitraires différents. Les jetons de raisonnement consommés sont en revanche relevés et rapportés, ce qui rend l'écart mesurable au lieu d'être supposé.

1.2.4.4 Limites assumées

- La température n'est pas contrôlable sur tous les modèles : cinq des dix — les deux Claude, les deux GPT et Kimi K3 — sont évalués à la valeur par défaut de leur fournisseur.
- L'effort de raisonnement n'est pas égalisé entre éditeurs, faute d'échelle commune.
- La dégradation en contexte long n'est pas mesurée : avec 42 échantillons par modèle, les effectifs par tranche de longueur seraient trop faibles pour conclure.
- Deux éditeurs sur six ne sont représentés que par un modèle — Moonshot et Alibaba : l'effet de gamme n'y est pas mesurable.

1.2.5 Évaluation et synthèse

L'évaluation part de l'étiquetage brut, avant toute agrégation. Le tableau 1.11 rapporte, modèle par modèle et niveau par niveau, les effectifs des catégories définies en section 1.2.2.2

— VP, FP, FN dont abstentions (Ab), identifiants obsolètes (OBS) et hallucinations (H) — auxquels s’ajoute le compteur de mauvaise matrice (MM) de la section 1.2.2.5, c’est-à-dire les identifiants pris hors de la matrice de l’échantillon.

Modèle	Niveau	VP	FP	FN	dont			
					Ab	OBS	H	MM
Claude Opus 4.8	Tactique (n = 126)	74	52	52	0	0	0	0
	Technique (n = 126)	71	55	55	0	1	0	0
	Sous-technique (n = 66)	42	24	24	0	3	0	0
Claude Sonnet 5	Tactique (n = 126)	64	60	62	2	0	0	0
	Technique (n = 126)	67	57	59	2	3	0	0
	Sous-technique (n = 66)	40	25	26	1	3	0	0
GPT-5.6 Sol	Tactique (n = 126)	82	44	44	0	0	0	0
	Technique (n = 126)	76	50	50	0	2	0	0
	Sous-technique (n = 66)	42	24	24	0	3	0	0
GPT-5.6 Luna	Tactique (n = 126)	63	62	63	1	0	13	4
	Technique (n = 126)	66	59	60	1	2	0	5
	Sous-technique (n = 66)	42	24	24	0	3	0	0
Gemini 3.1 Pro	Tactique (n = 125)	70	55	55	0	0	1	0
	Technique (n = 125)	60	65	65	0	3	0	1
	Sous-technique (n = 65)	42	23	23	0	3	0	0

Modèle	Niveau	VP	FP	FN	dont			
					Ab	OBS	H	MM
Gemini 3.8 Flash	Tactique (n = 125)	81	44	44	0	0	0	0
	Technique (n = 125)	75	50	50	0	3	0	0
	Sous-technique (n = 66)	44	22	22	0	3	0	0
Kimi K3	Tactique (n = 126)	68	58	58	0	0	1	3
	Technique (n = 126)	71	55	55	0	2	0	0
	Sous-technique (n = 66)	41	25	25	0	3	0	0
DeepSeek V4 Pro	Tactique (n = 126)	81	45	45	0	0	2	4
	Technique (n = 126)	72	54	54	0	2	0	1
	Sous-technique (n = 66)	44	22	22	0	3	0	0
DeepSeek V4 Flash	Tactique (n = 126)	75	51	51	0	0	3	3
	Technique (n = 126)	76	50	50	0	2	0	1
	Sous-technique (n = 66)	41	25	25	0	3	0	0
Qwen3.8	Tactique (n = 126)	62	64	64	0	0	3	6
	Technique (n = 126)	72	54	54	0	0	0	1
	Sous-technique (n = 66)	42	24	24	0	3	0	0

TABLEAU 1.11 – Étiquetage des prédictions des dix modèles. Entre parenthèses, le nombre d'exécutions évaluées au niveau considéré. FP est la valeur de la base stricte : elle comprend les colonnes OBS et H, reportées à part comme sous-comptes ; FN = FP + dont Ab.

Les hallucinations n'apparaissent qu'au niveau tactique, et sur six modèles ; GPT-5.6 Luna en concentre treize à lui seul. Ces six modèles sont exactement ceux qui se trompent de matrice : les deux Claude, GPT-5.6 Sol et Gemini 3.8 Flash n'ont aucun identifiant fabriqué ni hors matrice. L'abstention reste marginale, limitée à Claude Sonnet 5 et GPT-5.6 Luna. Les identifiants obsolètes, indétectables au niveau tactique, se concentrent au niveau sous-technique, où neuf modèles sur dix en produisent exactement trois. À ce même niveau, les dix modèles se tiennent entre 40 et 44 bonnes réponses sur 66 : c'est celui où ils se distinguent le moins.

Un score unique masquerait les compromis : un modèle rapide peut être bavard, un modèle précis peut s'abstenir souvent, un modèle exact peut s'arrêter trop tôt dans la hiérarchie.

Les modèles LLM seront comparés entre eux sur ces différents axes :

- **Qualité du mapping et actualité des connaissances** — rapportée en deux bases, stricte et tolérante, et ordonnée par gravité décroissante de la faute : le mauvais choix de matrice, le taux d'hallucination, le F1 par niveau — technique, tactique, sous-technique —, le taux d'obsolescence, puis le taux d'abstention.
- **Fiabilité, temps de traitement et coût** — ordonnée dans cet ordre : la stabilité, la latence médiane, le coût en jetons, puis l'efficacité.

Comment le premier axe est classé. L'ordre des critères suit la gravité de la faute, et non sa fréquence. Le mauvais choix de matrice vient en tête : un identifiant Enterprise proposé sur un journal ICS — ou l'inverse — ne désigne pas seulement une mauvaise technique, il désigne un mauvais univers, et la mitigation qui en découle est inapplicable au système observé. Il est compté en effectif brut, les trois niveaux confondus, parce qu'un seul de ces identifiants suffit à rendre la réponse inutilisable. Vient ensuite le taux d'hallucination : un identifiant fabriqué oriente l'analyste vers une entrée qui n'existe pas, et un modèle qui en produit est écarté d'un usage opérationnel quel que soit son F1. Le F1 ne départage donc que les modèles ayant passé ces deux filtres ; il est lu niveau par niveau, la technique d'abord — les mitigations MITRE s'y rattachent —, puis la tactique, puis la sous-technique. Le taux d'obsolescence et le taux d'abstention ne servent qu'à trancher les égalités restantes.

Le tri est lexicographique : chaque critère ne départage que les égalités laissées par le précédent, et aucune grandeur composite n'est fabriquée — chaque colonne du tableau reste une métrique définie en section 1.2.3. Le même ordre est appliqué aux deux bases, de sorte que la seule différence entre les tableaux 1.12 et 1.13 soit le traitement des identifiants obsolètes : comptés comme des erreurs en base stricte, crédités en base tolérante. L'écart entre les deux mesure l'actualité des connaissances du modèle, et rien d'autre.

Comment le second axe est classé. La stabilité vient en premier : un modèle qui ne rend pas deux fois la même réponse au même journal n'est pas exploitable dans un centre opérationnel,

quelle que soit sa vitesse. La latence médiane (p50) suit, comme temps de traitement courant ; le 95e centile est rapporté à titre de pire cas ordinaire mais n’entre pas dans le tri, un seul appel lent ne devant pas déclasser un modèle par ailleurs régulier. Le coût vient ensuite, mesuré par les jetons moyens par exécution. L’efficacité — F1 technique strict par millier de jetons — est placée en dernier parce qu’elle est le seul indicateur de ce tableau à dépendre de la base de calcul : elle passe par l’étiquetage, alors que la stabilité, la latence et les jetons n’en dépendent pas. La sur-spécification ne sert que de départage ultime.

Dans les deux axes, les critères sont donc appliqués l’un après l’autre, dans l’ordre annoncé : un modèle ne peut pas compenser une faute grave par une bonne performance sur un critère plus fin.

1.2.5.1 *Qualité du mapping et actualité des connaissances*

Les tableaux 1.12 et 1.13 appliquent l’ordre de critères annoncé. Ils portent sur les mêmes exécutions et ne diffèrent que par le traitement des identifiants obsolètes ; les deux premiers critères — mauvaise matrice et hallucination — ne passent pas par la base de calcul et sont donc identiques dans les deux.

Rang	Modèle	Mauv.	Halluc.	F1	F1	F1 s.-t.	Obsol.	Abst.
		mat.		tech.	tact.			
1	GPT-5.6 Sol	0	0,0 %	0,603	0,651	0,636	1,6 %	0,0 %
2	Gemini 3.8 Flash	0	0,0 %	0,600	0,648	0,667	1,9 %	0,0 %
3	Claude Opus 4.8	0	0,0 %	0,563	0,587	0,636	1,3 %	0,0 %
4	Claude Sonnet 5	0	0,0 %	0,536	0,512	0,611	1,9 %	1,6 %
5	Gemini 3.1 Pro	1	0,3 %	0,480	0,560	0,646	1,9 %	0,0 %
6	Kimi K3	3	0,3 %	0,563	0,540	0,621	1,6 %	0,0 %
7	DeepSeek V4 Flash	4	0,9 %	0,603	0,595	0,621	1,6 %	0,0 %
8	DeepSeek V4 Pro	5	0,6 %	0,571	0,643	0,667	1,6 %	0,0 %
9	Qwen3.8	7	0,9 %	0,571	0,492	0,636	0,9 %	0,0 %
10	GPT-5.6 Luna	9	4,1 %	0,526	0,502	0,636	1,6 %	0,6 %

TABLEAU 1.12 – Classement de la qualité du mapping, base stricte.

Les hallucinations sont un problème d’homonymie entre matrices. Les vingt-trois identifiants fabriqués du panel sont tous des tactiques, et tous sur des échantillons ICS. Ils ne relèvent pas d’une invention arbitraire mais d’un défaut de repérage entre les deux matrices, dont l’homonymie est la cause. Neuf tactiques portent en effet un nom identique de part et d’autre : Collection (TA0009 / TA0100), Command and Control (TA0011 / TA0101), Discovery (TA0007 / TA0102), Execution (TA0002 / TA0104), Impact (TA0040 / TA0105), Initial Access (TA0001 / TA0108), Lateral Movement (TA0008 / TA0109), Persistence (TA0003 / TA0110) et Privilege Escalation (TA0004 / TA0111). Le modèle reconnaît l’intention, mais doit encore choisir l’univers — et c’est là qu’il échoue, de deux façons : soit il donne l’identifiant Enterprise homonyme, soit il prolonge la numérotation ICS au-delà de TA0111, dernière tactique réelle, produisant TA0113 à TA0119.

Un exemple où l’erreur est entièrement imputable à l’homonymie. L’échantillon S012 — exécution d’un script sur l’hôte PLC, tactique attendue TA0104 Execution, matrice ICS — l’illustre sans ambiguïté. Sept modèles répondent TA0104 aux trois exécutions. GPT-5.6 Luna répond TA0002, l’Execution d’Enterprise, aux trois exécutions ; Qwen3.8 fait de même deux fois sur trois. Aucun des deux ne s’est trompé d’intention : ils ont correctement identifié une exécution de code, puis désigné la mauvaise matrice. Trois autres échantillons présentent le même profil — S000 et S001 pour Discovery, S003 pour Collection — et les trois modèles concernés sont exactement ceux que le premier critère élimine.

Conséquence opérationnelle. La mitigation attachée à TA0002 vise des systèmes d’information ; celle de TA0104 vise des automates. Un identifiant juste de nom mais faux d’univers oriente l’analyste vers un catalogue de contre-mesures inapplicable au système observé. C’est pourquoi le mauvais choix de matrice précède l’hallucination dans l’ordre des critères, et l’hallucination le F1.

Première étape : l’élimination par la matrice. Six modèles sur dix produisent au moins un identifiant pris hors de la matrice de l’échantillon et sont écartés avant même que leur F1 soit lu : Gemini 3.1 Pro (1), Kimi K3 (3), DeepSeek V4 Flash (4), DeepSeek V4 Pro (5), Qwen3.8 (7) et GPT-5.6 Luna (9), soit vingt-neuf identifiants au total. Le cas de DeepSeek V4 Flash est le plus démonstratif : son F1 technique, 0,603, égale celui du premier du tableau, et il termine pourtant septième. C’est exactement l’effet recherché par l’ordre des critères — une réponse prise dans le mauvais univers ne se rattrape pas par un bon score ailleurs, puisque la mitigation qu’elle entraîne ne s’applique pas au système observé.

Deuxième étape : l’hallucination. Elle ne départage personne de plus ici, les six modèles concernés ayant déjà été écartés par la matrice, mais elle confirme la dernière place de GPT-5.6 Luna : ses 4,1 % — treize identifiants fabriqués sur les vingt-trois du panel — représentent plus de quatre fois le pire des neuf autres modèles, tous sous 1 %.

Troisième étape : le F1. Il n'arbitre donc qu'entre les quatre modèles qui franchissent les deux premiers filtres sans rien concéder : GPT-5.6 Sol, Gemini 3.8 Flash, Claude Opus 4.8 et Claude Sonnet 5. En base stricte, GPT-5.6 Sol prend la tête d'un souffle sur le F1 technique (0,603 contre 0,600), les deux Claude suivant à 0,040 et 0,067.

Rang	Modèle	Mauv. mat.	Halluc.	F1 tech.	F1 tact.	F1 s.-t.	Obsol.	Abst.
1	Gemini 3.8 Flash	0	0,0 %	0,624	0,648	0,712	1,9 %	0,0 %
2	GPT-5.6 Sol	0	0,0 %	0,619	0,651	0,682	1,6 %	0,0 %
3	Claude Opus 4.8	0	0,0 %	0,571	0,587	0,682	1,3 %	0,0 %
4	Claude Sonnet 5	0	0,0 %	0,560	0,512	0,656	1,9 %	1,6 %
5	Gemini 3.1 Pro	1	0,3 %	0,504	0,560	0,692	1,9 %	0,0 %
6	Kimi K3	3	0,3 %	0,579	0,540	0,667	1,6 %	0,0 %
7	DeepSeek V4 Flash	4	0,9 %	0,619	0,595	0,667	1,6 %	0,0 %
8	DeepSeek V4 Pro	5	0,6 %	0,587	0,643	0,712	1,6 %	0,0 %
9	Qwen3.8	7	0,9 %	0,571	0,492	0,682	0,9 %	0,0 %
10	GPT-5.6 Luna	9	4,1 %	0,542	0,502	0,682	1,6 %	0,6 %

TABLEAU 1.13 – Classement de la qualité du mapping, base tolérante.

Les identifiants obsolètes se concentrent sur trois attaques. Trois échantillons concentrent l'intégralité des identifiants révoqués. S027 — effacement des journaux système Linux — est mappé T1070.002 par les dix modèles, aux trois exécutions : cet identifiant a été révoqué au profit de T1685.006, qui porte exactement le même nom. S007 — message Modbus non autorisé — est mappé T0855 par huit modèles sur dix, révoqué au profit de T1692.001. S038 — clés de registre d'exécution automatique — n'est touché que par GPT-5.6 Sol, qui propose T1060, révoqué au profit de T1547.001. Dans les trois cas le raisonnement est juste et seul le numéro a changé : c'est précisément ce que la base tolérante crédite.

Comparaison sur les trois modèles retenus. Sur S027, les trois se comportent à l'identique : T1070.002 aux trois exécutions, soit un faux positif au niveau technique et un obsolète au niveau sous-technique. Sur S007, Gemini 3.8 Flash propose T0855 aux trois exécutions — donc crédité en base tolérante — tandis que GPT-5.6 Sol propose T0836, une technique vivante mais fausse, jamais rattrapable ; Claude Opus 4.8 est entre les deux, T0836 deux fois

et T0855 une fois. Sur S038, Claude Opus 4.8 et Gemini 3.8 Flash répondent juste aux trois exécutions, GPT-5.6 Sol deux fois T1060 puis une fois la bonne réponse. L'écart entre base stricte et base tolérante ne mesure donc pas une différence de raisonnement, mais la fraîcheur du référentiel interne.

Ce que l'ancrage par MCP change, sur Claude Opus 4.8. Rejouée avec le serveur MCP, la même campagne corrige les deux échantillons où le modèle était en défaut. Sur S027, les trois exécutions passent de T1070/T1070.002 à T1685/T1685.006 ; sur S007, de T0836 ou T0855 à T1692/T1692.001. S038, déjà juste, le reste. Le taux d'obsolescence tombe de 1,3 % à 0 % et le F1 progresse à tous les niveaux : 0,587 à 0,802 en tactique, 0,563 à 0,706 en technique, 0,636 à 0,758 en sous-technique. L'ancrage ne rend pas le modèle plus perspicace, il lui donne le référentiel courant.

Les hallucinations sont un phénomène tactique et exclusivement ICS. Les 23 hallucinations du panel sont toutes au niveau tactique, et toutes sur des échantillons ICS. Les identifiants fabriqués — TA0113 à TA0119, plus TA0089 et TA0024 — se situent juste au-delà de la plage réelle des tactiques ICS, qui s'arrête à TA0111. Les modèles extrapolent la numérotation au lieu de la connaître : ils savent que les tactiques ICS commencent à TA0100 et poursuivent la série. Les tactiques concernées sont Impair Process Control (S006, S007, S008), Collection (S002, S004, S005), Discovery (S001), Inhibit Response Function (S009, S010) et Persistance (S013) — soit les tactiques propres à l'univers industriel, celles qui n'ont pas d'équivalent Enterprise dont le modèle pourrait se souvenir.

L'homonymie entre matrices élimine des modèles par ailleurs corrects. Neuf tactiques portent le même nom dans les deux matrices : Collection, Command and Control, Discovery, Execution, Impact, Initial Access, Lateral Movement, Persistance et Privilege Escalation. Un modèle qui reconnaît correctement l'intention mais ignore qu'il lit un journal industriel produit l'identifiant Enterprise homonyme — TA0007 au lieu de TA0102 pour Discovery, TA0009 au lieu de TA0100 pour Collection. Quatre échantillons sont touchés : S000 et S001 (Discovery), S003 (Collection) et S012 (Execution). Trois modèles commettent cette confusion : Qwen3.8 quatre fois, DeepSeek V4 Pro et GPT-5.6 Luna trois fois chacun. Le raisonnement est bon, l'univers est faux, et la mitigation qui en découlerait serait inapplicable au système observé — d'où son rang de premier critère d'élimination.

Le cas de Claude Opus 4.8. Claude Opus 4.8 ne produit aucun identifiant hors matrice ni aucune hallucination, avec ou sans serveur. Il ne doit donc pas son maintien dans le panel à l'ancrage, mais à une connaissance correcte de la frontière entre les deux univers. L'apport du MCP porte ailleurs, sur l'actualité du référentiel.

Les tactiques concernées par l'obsolescence chez Claude Opus 4.8. Deux tactiques du corpus concentrent les identifiants obsolètes de Claude Opus 4.8, et serviront d'exemples au

chapitre 2. TA0112 Defense Impairment, matrice Enterprise, par l'échantillon S027 — effacement des journaux système Linux — où T1070.002 est proposé aux trois exécutions alors que l'identifiant courant est T1685.006. Et TA0106 Impair Process Control, matrice ICS, par l'échantillon S007 — message Modbus non autorisé — où T0855 est proposé une fois sur trois alors que l'identifiant courant est T1692.001. Les deux cas relèvent d'une révocation avec successeur officiel, donc résolubles par simple consultation du référentiel.

Modèle	Enterprise	ICS	Total O	Affirm.	Taux
Claude Opus 4.8	3	1	4	318	1,3 %
Claude Sonnet 5	3	3	6	313	1,9 %
GPT-5.6 Sol	5	0	5	318	1,6 %
GPT-5.6 Luna	3	2	5	316	1,6 %
Gemini 3.1 Pro	3	3	6	315	1,9 %
Gemini 3.8 Flash	3	3	6	316	1,9 %
Kimi K3	3	2	5	318	1,6 %
DeepSeek V4 Pro	3	2	5	318	1,6 %
DeepSeek V4 Flash	3	2	5	318	1,6 %
Qwen3.8	3	0	3	318	0,9 %
Total	32	18	50	3 168	1,6 %

TABLEAU 1.14 – Répartition des identifiants obsolètes par matrice et taux d'obsolescence, par modèle.

Lecture du tableau. Les 32 identifiants obsolètes Enterprise proviennent presque tous du même échantillon, S027, sur lequel les dix modèles se trompent aux trois exécutions ; GPT-5.6 Sol y ajoute S038. Les 18 identifiants ICS proviennent tous de S007. L'obsolescence n'est donc pas diffuse : elle se concentre sur les rares techniques renommées entre deux versions, et frappe indifféremment les dix modèles. Aucun ne se distingue par un référentiel plus à jour — Qwen3.8 affiche le taux le plus bas, 0,9 %, non parce qu'il connaît T1685.006, mais parce qu'il se trompe autrement sur S007.

La base tolérante ne déplace que la tête du classement. Gemini 3.8 Flash passe devant GPT-5.6 Sol (0,624 contre 0,619 en technique) : une part plus grande de ses erreurs était constituée d'identifiants révoqués dont le successeur officiel était la bonne réponse. Les quatre modèles retenus restent les mêmes, et l'élimination ne bouge pas, puisque ni la matrice ni l'hallucination ne dépendent de la base. Le gain au passage en base tolérante mesure l'actualité du référentiel interne de chaque modèle : de 0,008 pour Claude Opus 4.8 à 0,024 pour Gemini 3.8 Flash et Claude Sonnet 5. Qwen3.8 est le seul dont le F1 technique ne bouge

pas d'un millièm — son référentiel est le plus à jour du panel —, mais ses sept identifiants hors matrice l'ont déjà écarté.

Ce que ces deux tableaux servent à la comparaison avec le MCP. Ils constituent la ligne de base de la phase 2 : mêmes échantillons, même protocole, mêmes modèles, mais sans serveur. Trois lectures en découlent. D'abord, c'est la base tolérante qui sera la référence à battre : la résolution des identifiants révoqués est précisément la fonction que le serveur mécanise, et comparer les résultats avec MCP à la seule base stricte confondrait deux effets — le rattrapage de version, déjà mesuré par l'écart entre les tableaux 1.12 et 1.13, et l'apport propre de l'ancrage, c'est-à-dire consulter le référentiel avant de répondre. Tout dépassement du tableau 1.13 mesurera l'ancrage, et non le simple rafraîchissement des connaissances. Ensuite, les deux colonnes de tête deviennent les indicateurs directs de l'apport du serveur : celui-ci ne sert que des identifiants vivants, pris dans la matrice interrogée, de sorte que les vingt-neuf identifiants hors matrice et les vingt-trois identifiants fabriqués relevés ici doivent disparaître avec lui ; s'ils subsistent, c'est que l'ancrage n'a pas été utilisé. Enfin, l'écart entre les deux zones désigne le terrain où le gain est attendu : au niveau technique, les dix modèles se tiennent entre 0,639 et 0,750 sur la matrice Enterprise, mais entre 0,167 et 0,452 sur la matrice ICS. La connaissance du domaine industriel est la plus datée et la plus lacunaire ; c'est là que le référentiel outillé a le plus à apporter.

1.2.5.2 Fiabilité, temps de traitement et coût

Aucun indicateur du tableau 1.15 ne dépend de la base de calcul, à l'exception de la dernière colonne : l'efficacité rapporte le F1 technique strict au coût en jetons.

Rang	Modèle	Stab.	p50 (ms)	p95 (ms)	Jetons	Sur- sp.	N1	N2	Effic.
1	Gemini 3.1 Pro	88,1 %	18 532	103 824	9 023	25,0 %	0 %	0 %	0,053
2	Gemini 3.8 Flash	81,0 %	10 664	44 482	7 566	10,2 %	0 %	0 %	0,079
3	Claude Opus 4.8	76,2 %	4 282	6 050	6 572	10,0 %	0 %	0 %	0,086
4	Claude Sonnet 5	66,7 %	4 291	10 994	6 740	15,0 %	0 %	0 %	0,080
5	GPT-5.6 Sol	64,3 %	6 086	24 145	4 649	10,0 %	0 %	0 %	0,130
6	Kimi K3	59,5 %	24 034	202 504	6 027	10,0 %	0 %	0 %	0,094

Rang	Modèle	Stab.	p50 (ms)	p95 (ms)	Jetons	Sur- sp.	N1	N2	Effic.
7	Qwen3.8	57,1 %	82 854	269 617	9 641	11,7 %	0 %	0 %	0,059
8	GPT-5.6 Luna	54,8 %	4 626	28 758	5 004	26,7 %	0 %	0 %	0,105
9	DeepSeek V4 Flash	52,4 %	3 407	19 812	5 594	11,7 %	0 %	0 %	0,108
10	DeepSeek V4 Pro	42,9 %	40 724	151 756	7 941	18,3 %	0 %	0 %	0,072

TABLEAU 1.15 – Classement de la fiabilité, du temps de traitement et du coût.

Deux modèles écartés avant la campagne. Deux modèles ont été écartés après essais. kimi-k2.7-code affichait 158 s de latence médiane, soit plus de cinq heures de campagne contre 50 minutes pour kimi-k3 de la même lignée. qwen3.5-35b-a3b consommait 47 626 jetons par appel, dont 94 % en raisonnement, avec deux sorties non conformes sur huit essais. Dans les deux cas, la dépense n’était pas justifiée par la qualité. Ce n’est pas un effet de taille — DeepSeek V4 Flash, modèle léger, consomme moins que DeepSeek V4 Pro tout en obtenant de meilleurs résultats — mais de longueur de raisonnement.

Pourquoi aucun niveau de raisonnement n’est imposé. L’effort de raisonnement est laissé aux valeurs par défaut et n’est pas traité comme facteur : aucune échelle commune n’existe entre éditeurs, et forcer un niveau « maximal » partout déplacerait les modèles vers des points arbitraires différents plutôt que comparables. Même logique que pour la température. Le faire varier changerait la question posée et porterait la campagne à 3 780 appels. L’effet reste mesuré : les jetons de raisonnement sont relevés et rapportés — c’est ce qui a permis d’écarter les deux modèles ci-dessus.

La stabilité ne mesure pas la justesse. Gemini 3.1 Pro affiche la meilleure du panel, 88,1 %, tout en étant dernier en F1 technique en base stricte : il se trompe de façon remarquablement reproductible. Sa sur-spécification de 25,0 % — une sous-technique inventée sur un quart des techniques terminales — et sa médiane de 18,5 s achèvent de le disqualifier, et il avait de toute façon été écarté par la matrice.

Parmi les quatre modèles retenus, la hiérarchie s’inverse d’un critère à l’autre. Gemini 3.8 Flash est le plus stable (81,0 %), devant Claude Opus 4.8 (76,2 %), Claude Sonnet 5 (66,7 %) et GPT-5.6 Sol (64,3 %). Sur la vitesse, l’ordre se renverse presque : les deux Claude traitent un échantillon en 4,3 s de médiane, GPT-5.6 Sol en 6,1 s, Gemini 3.8 Flash en 10,7 s.

Le 95e centile sépare mieux que la médiane. Claude Opus 4.8 est à 6,0 s, le plus bas des dix modèles, soit 1,4 fois sa propre médiane ; Claude Sonnet 5 est à 11,0 s (2,6 fois), GPT-5.6

Sol à 24,1 s (4,0 fois) et Gemini 3.8 Flash à 44,5 s (4,2 fois). C'est ce rapport, plus que la médiane seule, qui dit si un centre opérationnel peut planifier sa charge : il mesure le pire cas ordinaire, celui qu'une moyenne masquerait.

Le coût sépare franchement. GPT-5.6 Sol consomme 4 649 jetons par exécution, Claude Opus 4.8 6 572, Claude Sonnet 5 6 740 et Gemini 3.8 Flash 7 566 — soit 63 % de plus que GPT-5.6 Sol. L'efficacité classe donc presque à l'inverse de la qualité brute : 0,130 pour GPT-5.6 Sol, 0,086 pour Claude Opus 4.8, 0,080 pour Claude Sonnet 5 et 0,079 pour Gemini 3.8 Flash.

Deux colonnes valent zéro pour les dix modèles. N1 et N2 sont neutralisées par le format de sortie, qui impose les trois champs : la sous-spécification est empêchée par le protocole, non par une vertu des modèles. Les colonnes sont conservées parce que cette nullité est elle-même un résultat.

1.2.5.3 *Choix du modèle*

L'élimination par la matrice, puis par l'hallucination, ramène le panel à quatre candidats : GPT-5.6 Sol, Gemini 3.8 Flash, Claude Opus 4.8 et Claude Sonnet 5. Claude Sonnet 5 est le seul des quatre à être dominé sur tous les axes par un autre membre du groupe — Claude Opus 4.8 le devance en qualité, en stabilité, en régularité et en efficacité, pour un coût presque identique. Restent trois profils, chacun défendable sur un axe différent.

Gemini 3.8 Flash est le plus performant, et c'est sa consommation qui l'écarte. Il est premier en base tolérante, deuxième en base stricte, et meilleur du panel au niveau sous-technique (0,712). Mais il consomme 7 566 jetons par exécution pour un F1 technique qui n'excède celui de GPT-5.6 Sol que de 0,005 en base tolérante, et lui reste inférieur de 0,003 en base stricte : l'écart de qualité est nul au regard de la variabilité entre exécutions, alors que le surcoût, lui, est réel et systématique. Or les jetons ne se paient pas qu'en facturation, ils se paient aussi en temps : la génération étant séquentielle, chaque jeton supplémentaire allonge la réponse. D'où une médiane de 10,7 s, deux fois et demie celle de Claude Opus 4.8, et un 95e centile de 44,5 s, sept fois le sien. Dans un centre qui qualifie des milliers d'alertes par jour, ces trois mille jetons et ces six secondes de plus par mapping se multiplient par le volume. La phase 2 aggrave le calcul, puisque l'ancrage ajoute au contexte les appels d'outils et leurs résultats : un modèle déjà bavard sans serveur le sera davantage avec. Sa dernière place en efficacité parmi les quatre retenus, 0,079, résume l'objection — cette consommation n'achète pas de qualité, elle alourdit l'exploitation.

GPT-5.6 Sol se démarque par le rapport entre la qualité et le coût. Premier en base stricte, deuxième en base tolérante, il est aussi le plus sobre des quatre (4 649 jetons) et de loin le plus efficace (0,130, soit 65 % de plus que Gemini 3.8 Flash), avec une médiane de 6,1 s

— près de deux fois plus rapide que Gemini 3.8 Flash. Ce qui l'écarte tient à sa régularité : 64,3 % de stabilité, la plus basse des quatre, et un 95e centile à 24,1 s, quatre fois sa médiane. Un modèle qui ne rend pas le même triplet sur plus d'un échantillon sur trois complique la phase 2, car sa dispersion propre se mêlerait à l'effet que l'ancrage doit produire.

Claude Opus 4.8 est le juste milieu, et c'est le modèle retenu. Il n'est premier sur aucun critère isolé, mais il est le seul à ne céder franchement sur aucun. Sur la qualité, il est troisième dans les deux bases, à 0,040 du premier en F1 technique strict, sans mauvaise matrice ni hallucination, avec le taux d'obsolescence le plus bas du panel (1,3 %) et aucune abstention. Sur la fiabilité, il est le plus stable des trois candidats restants (76,2 %), le plus rapide en médiane (4 282 ms) et surtout le plus prévisible : son 95e centile de 6 050 ms est le plus bas des dix modèles, à 1,4 fois sa médiane, quand les deux autres tournent autour de quatre fois. Sur le coût, ses 6 572 jetons le placent au milieu et son efficacité de 0,086 devance celle de Gemini 3.8 Flash. C'est le profil qu'une ligne de base réclame : ce que la phase 2 mesure, c'est l'apport de l'ancrage, et un modèle dont la dispersion propre est faible laisse cet effet visible au lieu de le noyer. La marge qui lui reste — 0,563 de F1 technique, et surtout un effondrement de 0,714 sur Enterprise à 0,262 sur ICS — est précisément celle que le serveur MCP doit combler.

Le choix du modèle est donc arrêté : **Claude Opus 4.8** portera la phase 2, et les tableaux 1.12, 1.13 et 1.15 en constituent la ligne de base sans serveur, celle contre laquelle l'apport de l'ancrage sera mesuré.

CHAPITRE 2

L'APPROCHE MCP

2.1 INTRODUCTION

[Section à rédiger.]

2.2 LE SERVEUR MCP

MITRE ATT&CK est un catalogue public de comportements d'attaquants informatiques, publié sous la forme d'un fichier de données que chacun peut télécharger. Notre serveur MCP est le connecteur qui expose les matrices Enterprise et ICS de ce référentiel aux modèles d'intelligence artificielle. Voici comment le tout fonctionne.

2.2.1 Architecture : hôte, client et serveur

Le MCP relie trois rôles : l'hôte porte le modèle, le client y maintient une connexion un-à-un avec chaque serveur, et le serveur expose les outils que le modèle invoque — lequel ne s'adresse jamais directement au serveur (figure 2.1).

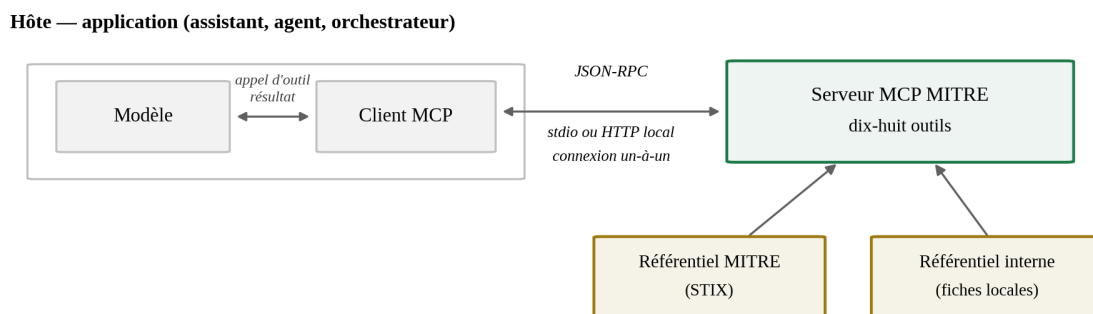


FIGURE 2.1 – Architecture MCP. Le modèle raisonne dans l'hôte ; le client porte la connexion ; le serveur répond à partir de ses deux sources, en lecture seule.

2.2.2 Les deux sources du serveur

Le serveur sert deux sources, l'une publique, l'autre propre à l'organisation.

Le référentiel MITRE. La base est chargée depuis les paquets STIX publiés par MITRE, où chaque élément est un objet — tactique, technique, sous-technique, mitigation, groupe,

logiciel, campagne — et où chaque lien est lui-même un objet (figure 2.2) : la technique appartient à une tactique de la *kill chain*, subtechnique-of rattache *LSASS Memory* à sa parente, mitigates relie une mesure de défense à la technique, uses relie un groupe à la sous-technique qu’il emploie, et revoked-by conduit d’un identifiant retiré à son remplaçant vivant. La fiche renvoyée par `get_technique` est assemblée en suivant ces liens : tactique, mitigations propres et héritées de la parente, groupes, sources de détection.

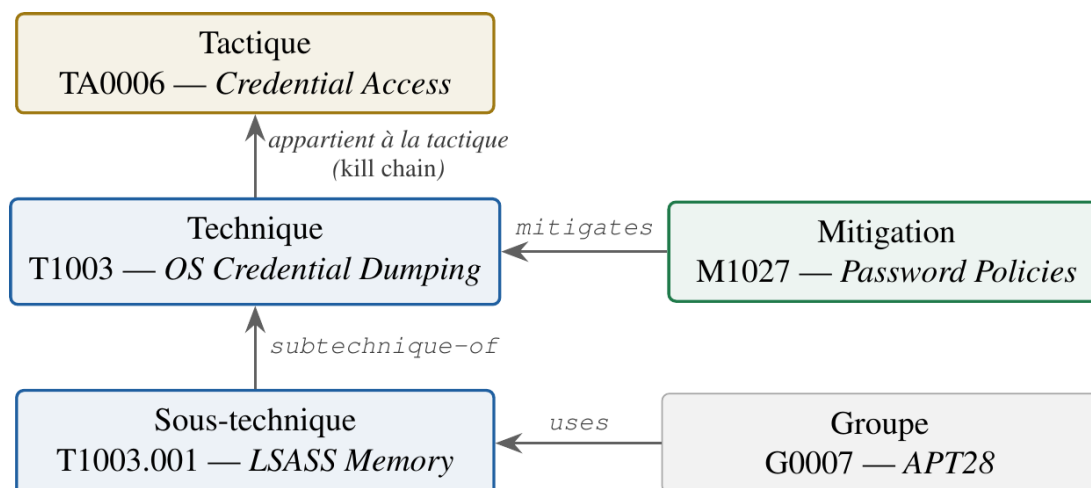


FIGURE 2.2 – Le référentiel MITRE tel que le serveur le lit : des objets reliés par des relations explicites.

Le référentiel interne. Un fichier JSON de fiches aux identifiants propres à l’organisation, chacune rattachée à une technique MITRE dont le statut est signalé. Voici une fiche réelle du référentiel :

```

{"id": "THQ0001",
 "name": "Phishing imitant le portail RH interne",
 "description": "Leurre imitant le portail RH pour voler des identifiants.",
 "technique_mitre_liee": "T1566.002",
 "tactique": "initial-access",
 "source_cti": "incident-2026-03",
 "statut": "valide"}
```

2.2.3 Le chemin d’une requête

Un analyste soumet un signal au modèle : « Comment interprètes-tu ce signal avec le référentiel MITRE ATT&CK ? », accompagné d’une trace où un processus inconnu lit la mémoire de `lsass.exe`. Le modèle décide d’interroger le serveur ; l’appel passe par le client MCP, qui porte la connexion. La figure 2.3 montre le chemin parcouru dans le serveur, et chaque étape

est développée ci-dessous en suivant cette même requête, de son émission à sa trace d'audit.

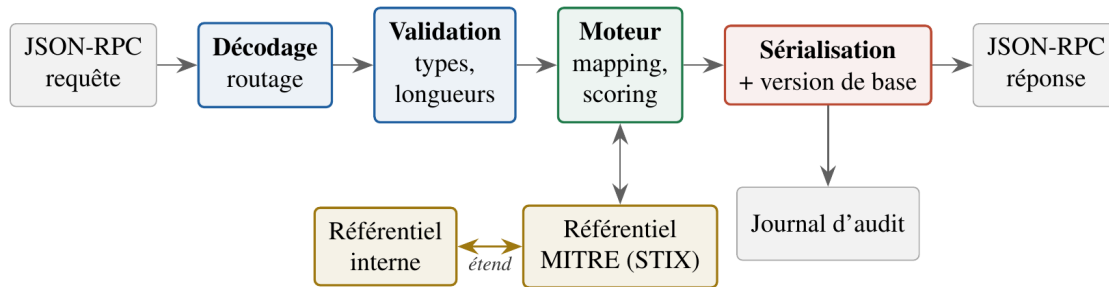


FIGURE 2.3 – Chemin d'une requête dans le serveur MCP.

- **Format des messages.** Les échanges suivent JSON-RPC 2.0, le format retenu par la spécification MCP : un objet JSON portant un identifiant, la méthode demandée et ses paramètres. Le serveur porte ce dialogue sur l'entrée et la sortie standard du processus, et sur un service HTTP local pour les clients qui ne savent pas parler ainsi ; les deux voies traversent le même code et ne changent rien à ce qui suit. Pour notre signal, le modèle formule une requête de recherche dans le vocabulaire ATT&CK et le client émet :

```
{ "jsonrpc": "2.0", "id": 3, "method": "tools/call",
  "params": { "name": "search_techniques",
    "arguments": { "query": "lsass credential dumping",
      "limit": 5 } } }
```

- **Décodage.** Le champ `method` détermine le traitement. Les noms de méthodes sont imposés par la spécification MCP et le dialogue typique en enchaîne trois : `initialize` (« bonjour, qui es-tu ? »), `tools/list` (« que sais-tu faire ? »), puis `tools/call` (« exécute `search_techniques` avec ce mot-clé »). Une quatrième, `ping`, peut être émise par l'un ou l'autre côté à tout moment : le destinataire doit répondre aussitôt par un résultat vide, ce qui atteste que la connexion est ouverte et le pair réactif ; sans réponse, l'émetteur la considère morte et la ferme. C'est par `tools/list`, en début de session, que le modèle apprend quels outils existent et quand les employer (figure 2.4) ; le tableau 2.1 en donne la liste. Une ligne illisible est journalisée puis écartée, sans bloquer les requêtes suivantes.

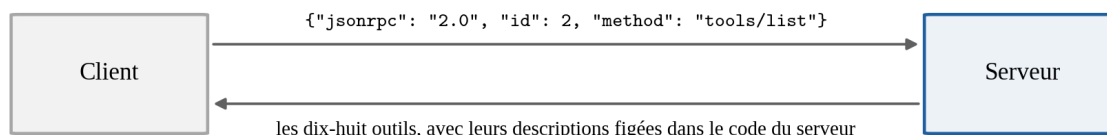


FIGURE 2.4 – L’énumération des outils : le modèle lit là les dix-huit descriptions, figées dans le code, qui guident ensuite son choix d’outil.

Outil	Description exposée au modèle
search_techniques	Recherche lexicale de techniques candidates ; une requête pour un comportement, résultats scorés avec leurs preuves
get_technique	Détail complet d’une technique pour confirmer un candidat : tactiques, plateformes, détections, sous-techniques, groupes, logiciels, campagnes, mitigations propres et de la parente, équipements ICS ; redirige un identifiant révoqué, signale un déprécié
get_tactics	Les tactiques Enterprise et ICS dans l’ordre de la <i>kill chain</i>
get_tactic	Une tactique et ses techniques, parentes et sous-techniques
get_mitigations	Les contre-mesures associées à une technique, pour produire une recommandation
get_mitigation	Une mesure de mitigation et toutes les techniques qu’elle couvre
get_groups	Groupes APT : les plus actifs, ou ceux qui emploient une technique donnée, directement ou en campagne
get_group	Détail d’un groupe : alias, techniques directes, techniques observées en campagne, logiciels, campagnes attribuées
get_software	Logiciels et maliciels : les plus employés, ou ceux associés à une technique
get_campaign	Détail d’une campagne : période, groupes attribués, techniques et logiciels observés
get_asset	Équipement ICS — automate, interface, unité distante, historien — et les techniques qui le visent
get_datasources	Techniques détectables depuis une source de journaux donnée
list_internal_techniques	Référentiel interne : liste complète et prochain identifiant libre, à appeler avant toute proposition de fiche

Outil	Description exposée au modèle
get_internal_technique	Détail d'une fiche interne, avec la technique MITRE liée et son statut
search_internal	Recherche lexicale dans le référentiel interne ; obligatoire avant de proposer une fiche, pour écarter un doublon
internal_reload	Relecture du référentiel interne sans redémarrer le serveur ; le fichier n'est jamais écrit
mitre_stats	Version du serveur, versions ATT&CK chargées et statistiques de la base
mitre_update	Retéléchargement des deux matrices, avec bascule seulement si les deux réussissent

TABLEAU 2.1 – Les dix-huit outils exposés et leur description, telle que le modèle la lit.

- **Un décodage qui ne se tait jamais.** Un client mal réglé ne doit pas produire un silence, car le silence laisse le modèle attendre une réponse qui ne viendra pas. Trois défauts courants sont donc traités explicitement. Un message étalé sur plusieurs lignes est assemblé dans un tampon borné à un mégaoctet, les accolades étant comptées hors des chaînes de caractères : un } présent dans un extrait de journal ne fait plus croire que le message est terminé. Ce qui reste illisible reçoit l'erreur -32700. Une enveloppe mal formée — message qui n'est pas un objet, params qui n'en est pas un, nom d'outil qui n'est pas du texte, arguments qui n'est pas un objet — est refusée avec le code correspondant, -32600 ou -32602. Enfin la version de protocole est négociée au lieu d'être répétée : le serveur ne rend la version demandée que s'il la connaît, et répond sinon la plus ancienne qu'il prend en charge, 2024-11-05, plutôt que d'en accepter une qu'il ne saurait pas parler. La règle est la même partout : toute requête portant un identifiant reçoit une réponse, y compris quand elle est fautive.
- **Aucun état conservé entre deux appels.** Le serveur ne retient ni session, ni historique, ni contexte de conversation, et n'exige aucun identifiant de session. Chaque message porte tout ce qu'il faut pour être traité, et plusieurs messages peuvent d'ailleurs voyager ensemble : un lot JSON-RPC reçoit un tableau de réponses, de sorte que trois outils demandés dans le même tour partent en un seul envoi. Deux conséquences : la mémoire de l'échange est portée par le client, et la même requête rejouée sur la même version de base rend exactement la même réponse, puisque le résultat ne dépend pas de ce qui a été demandé avant.
- **Validation.** Avant toute exécution, l'appel est contrôlé contre le schéma que l'outil déclare : l'outil existe, les arguments requis sont présents, aucun argument étranger

n'est passé, les valeurs sont du bon type et d'une longueur bornée. Un filtre mal orthographié est refusé avec la liste des valeurs admises, et non traité comme une recherche sans résultat : une liste vide signifierait « aucune correspondance » et fausserait la conclusion.

```
get_technique(id="T1003.001", matrix="ICS")
$\rightarrow$ argument inconnu : matrix (attendus : id)
```

```
search_techniques("phishing", matrix="entreprise")
$\rightarrow$ matrice 'entreprise' inconnue - valeurs acceptees :
  Enterprise, ICS (ou IT, OT, SCADA)
```

```
search_techniques("phishing", platform="windowz")
$\rightarrow$ platform inconnue 'windowz' : attendu une valeur parmi Containers,
  ESXi, IaaS, Identity Provider, Linux, Network Devices, None,
  Office Suite, PRE, SaaS, Windows, macOS
```

- Quand la correction est évidente, le serveur corrige et le dit plutôt que d'échouer : un "limit": "abc" retombe sur la valeur par défaut, avec la note limit invalide : valeur par défaut appliquée.
- **Moteur : proposer, puis vérifier.** Le moteur ne conclut pas, il propose. Il rend des techniques candidates, chacune avec un score et avec les mots qui l'ont fait remonter ; c'est ensuite au modèle de confirmer le candidat retenu par get_technique, puis de le confronter au référentiel interne. La section 2.2.4 détaille son fonctionnement.
- **Référentiel MITRE.** Le moteur y cherche les techniques candidates, puis y assemble la fiche complète de celle qui est retenue.
- **Référentiel interne.** Il y vérifie ensuite si l'organisation décrit déjà ce comportement sous son propre identifiant.
- **Sérialisation.** Le résultat est mis en forme en JSON compact et la version de la base servie y est jointe : la réponse est datée, donc rejouable. Voici le contenu renvoyé au client pour notre requête, avec les champs réels du serveur (enveloppe JSON-RPC omise, premier candidat seul) :

```
{"db_version": {"Enterprise": "19.2", "ICS": "19.2"},
  "query": "lsass credential dumping",
  "query_tokens": ["credenti", "dump", "lsass"],
  "filtres": {"matrix": "aucun", "platform": "aucun",
              "tactic": "aucun"},
  "seuils": {"fort": 0.6, "faible": 0.3},
  "result_count": 5,
  "total_matches": 172,
```

```

"results": [
  {"id": "T1003.001", "name": "LSASS Memory",
   "full_name": "OS Credential Dumping: LSASS Memory",
   "matrix": "Enterprise", "score": 0.894,
   "is_subtechnique": true, "parent_id": "T1003",
   "tactics": ["TA0006"],
   "matched": {"name": ["lsass"],
               "parent_name": ["credenti", "dump"]},
   "matched_in": ["name", "parent_name"],
   "snippet": "..."}, ...],
"note": "le candidat de tete est une SOUS-technique ; sa
        parente T1003 est aussi candidate - retenir T1003
        si l'evidence ne distingue pas la sous-technique"

```

- **Journal d'audit.** Vingt-quatre types d'événements y sont tracés, et pas seulement l'appel d'outil. D'abord la base elle-même : le téléchargement de chaque matrice avec sa durée et le numéro de version obtenu, puis son chargement, dont le champ source dit si elle vient du réseau ou du cache, puis le démarrage, qui fixe la version servie pendant toute la session. Le chargement du référentiel interne laisse une ligne de même nature, avec le nombre de fiches retenues, le nombre de fiches écartées et le prochain identifiant libre.

```

{"ts": "...", "run_id": "0a3585246427",
 "event": "download_complete", "matrix": "Enterprise",
 "duration_ms": 2988, "release": "19.2"}
{"ts": "...", "run_id": "0a3585246427", "event": "db_loaded",
 "matrix": "Enterprise", "source": "cache", "release": "19.2",
 "techniques": 697}
{"ts": "...", "run_id": "0a3585246427",
 "event": "internal_db_loaded",
 "path": "referentiel_interne.json", "fiches": 2, "rejets": 0,
 "avertissements": 0, "next_id": "THQ0003"}
{"ts": "...", "run_id": "0a3585246427", "event": "server_start",
 "pid": 25184, "version": "2.2.1", "transport": "stdio",
 "db_version": {"Enterprise": "19.2", "ICS": "19.2"},
 "ready": true, "tools": 18}

```

- Ensuite le protocole : initialize consigne la version demandée par le client, celle qui a été négociée et l'identité du client; tools/list est consigné aussi, parce que c'est par cet appel que le modèle lit les descriptions d'outils sur lesquelles il fonde ses choix; et un appel adressé à un outil qui n'existe pas produit une ligne dont le statut vaut unknown_tool, car c'est la signature d'un outil inventé par le modèle et il faut pouvoir la compter.

- Enfin l'appel lui-même. Chaque étape franchie ou refusée écrit une ligne horodatée portant l'identifiant d'exécution de la session. La figure 2.5 montre la ligne produite par notre appel : l'outil invoqué, le statut, la durée, les arguments tronqués et un résumé du résultat — jamais le résultat entier. L'écriture est confiée à un fil d'exécution séparé, de sorte qu'elle ne retarde jamais une réponse, et si la file d'attente sature, une ligne publie le nombre d'événements perdus plutôt que de les taire.

```
{
  "ts": "...",
  "run_id": "0a3585246427",
  "event": "tool_call",
  "transport": "stdio",
  "req_id": 3,
  "tool": "search_techniques",
  "status": "success",
  "duration_ms": 7,
  "db_version": {
    "Enterprise": "19.2",
    "ICS": "19.2"
  },
  "args": {
    "query": "lsass credential dumping",
    "limit": "5"
  },
  "result": {
    "bytes": 2594,
    "result_count": 5,
    "total_matches": 172,
    "top_score": 0.894,
    "top_id": "T1003.001",
    "note": "..."}
}
```

FIGURE 2.5 – Ligne d'audit écrite pour cet appel. En bleu, l'identifiant qui corréle la session ; en vert, la durée et la version qui datent la réponse ; en ocre, les arguments tronqués ; en rouge, le résumé du résultat.

Le même échange se lit ainsi à trois instants : la requête émise, la réponse datée, la trace d'audit — trois états successifs du serveur pour une seule question d'analyste.

2.2.4 Le moteur de mapping

Le moteur est la partie du serveur qui transforme un signal en identifiants ATT&CK. Il travaille en deux temps, visibles en figure 2.6 : il propose des techniques candidates avec un score, puis il fournit de quoi les vérifier. Il ne tranche jamais à la place de l'analyste, et le rejet est une sortie prévue.

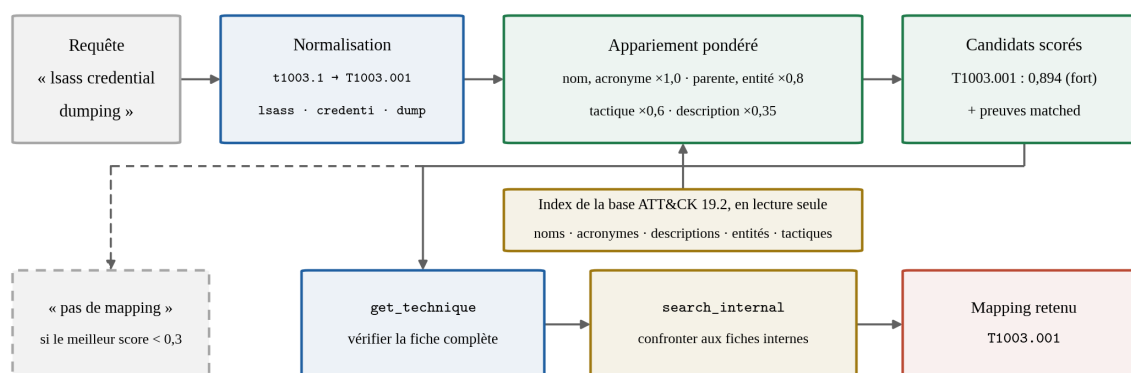


FIGURE 2.6 – Le moteur de mapping : proposer (chaîne du haut), puis vérifier et confronter (chaîne du bas), avec le rejet comme sortie légitime.

Ce que le moteur lit dans le référentiel. MITRE a changé sa façon de décrire la détection entre ses versions 17.1 et 18. Dans l'ancien modèle, chaque technique portait deux champs

de texte : une liste de sources de données et un paragraphe de détection. Dans le nouveau, l'information est publiée sous forme d'objets reliés, sur trois niveaux : une stratégie de détection dit quoi surveiller, un ou plusieurs *analytics* la mettent en œuvre, et chaque *analytic* nomme les journaux à consulter. Sur T1003.001, cela donne :

```
DET0363  Detection of Credential Dumping from LSASS Memory
          via Access and Dump Sequence
Analytic 1030  (Windows)
  journaux    WinEventLog:Sysmon:EventCode=10,
              WinEventLog:Security:EventCode=4673, ...
  composants  Process Access, Process Creation,
              File Creation, ...
```

Le serveur lit les deux modèles et annonce lequel il a trouvé, dans le champ `detection_model`. Sur la version 19.2, ce champ vaut `strategies` : les 697 techniques Enterprise et les 97 techniques ICS ont toutes au moins une stratégie de détection, et 652 d'un côté, 85 de l'autre portent des journaux exploitables, ce qui alimente un index de 692 libellés de sources. Si l'on place dans le cache le paquet d'une version 17.1, le champ vaut `hérité` : aucune stratégie n'y est publiée, mais 639 techniques Enterprise et 71 techniques ICS portent quand même leurs sources de données, et T1003.001 restitue les 1 119 caractères de son ancien texte de détection. Une étude menée sur une version ancienne reste donc exploitable, et le serveur ne rend jamais une liste de détections vide en laissant croire que MITRE n'a rien publié.

Un identifiant écrit de plusieurs façons. MITRE ne connaît qu'une seule forme pour une sous-technique, à trois décimales. Les rapports, les tickets et les humains en écrivent d'autres. Le serveur les ramène à la forme officielle avant toute consultation, dans tous les outils qui prennent un identifiant, et aussi pour les identifiants repérés à l'intérieur du texte d'une requête :

```
get_technique("t1003.1")    $\rightarrow$ T1003.001    LSASS Memory
get_technique("T1003.01")   $\rightarrow$ T1003.001    LSASS Memory
get_technique("T1003.001")  $\rightarrow$ T1003.001    LSASS Memory
```

Les identifiants d'anciennes techniques. Le référentiel bouge : des techniques sont retirées à chaque version. Un rapport de renseignement écrit il y a cinq ans, une règle de détection ou une fiche interne peuvent encore citer un identifiant qui n'existe plus. Deux situations se présentent, et elles appellent des gestes opposés.

La première : la technique a été remplacée. Le serveur suit la relation de remplacement publiée par MITRE et redirige vers la technique vivante. La version 19.2 compte 158 redirections de ce type, dont aucune n'aboutit à une impasse. MITRE enchaîne parfois plusieurs

remplacements successifs, et le chemin complet est conservé, car un ancien document peut citer justement l'identifiant intermédiaire :

```
get_technique("T1073")
$\\rightarrow$ {"redirect": true, "requested": "T1073",
  "replaced_by": "T1574.001", "via": ["T1574.002"],
  "remplacant_vivant": true,
  "note": "'T1073' est revoquee : consulter T1574.001 via
    get_technique (chaine de revocation : T1574.002)"}

```

La seconde : la technique a été retirée sans remplaçante. Elles sont 24 dans la version 19.2. Le serveur le dit explicitement, au lieu de répondre « non trouvée » comme pour une faute de frappe :

```
get_technique("T1064")
$\\rightarrow$ {"deprecated": true, "requested": "T1064", "name": "Scripting",
  "note": "'T1064' (Scripting) est DEPRECIEE par MITRE sans
    rempalcant : ne plus l'utiliser dans un mapping ;
    chercher le comportement avec search_techniques"}

```

La distinction compte, parce que les deux cas ne se corrigent pas de la même façon : une faute de frappe se réécrit, un identifiant déprécié oblige à remapper le comportement. La recherche signale l'un et l'autre lorsque l'identifiant apparaît dans la requête.

La base est lue, jamais écrite. Aucun des dix-huit outils ne modifie le référentiel MITRE ni le référentiel interne. Le serveur n'écrit que deux choses sur le disque, et aucune n'est un contenu d'analyse : une copie des paquets publiés par MITRE, qui sert de cache, et le journal d'audit. Le cache est écrit par remplacement atomique, si bien qu'une interruption ne laisse jamais un fichier à moitié écrit. Le retéléchargement suit la même règle : les deux matrices sont récupérées et analysées d'abord, et la base en mémoire n'est remplacée qu'en cas de succès complet ; un échec laisse la base précédente en place et le signale dans la réponse. La bascule se fait sous verrou, de sorte qu'aucune requête ne peut voir une base à moitié échangée. Le référentiel interne obéit au même principe : le serveur le relit, mais n'y écrit pas, et la création comme la validation d'une fiche restent des actes humains, faits dans le fichier JSON versionné par l'équipe. Pour le mapping, la conséquence est directe : tout identifiant renvoyé vient de la base chargée, et le modèle n'a pas la possibilité d'en composer un qui n'existe pas.

Le score. La recherche est lexicale : elle compare des mots, pas des sens. Le calcul se fait en quatre étapes. D'abord la requête est découpée en mots et chaque mot est ramené à sa racine : « lsass credential dumping » devient lsass, credenti, dump. La même opération est appliquée au référentiel au chargement, sans quoi dumping dans la requête et dump dans

le catalogue ne se rencontreraient jamais. Les mots de moins de trois lettres sont écartés, sauf s'ils contiennent un chiffre : c2, 2fa et t1003 sont retenus, to, of et or ne le sont pas. La raison est mesurable : ces trois mots figurent respectivement dans 790, 704 et 662 des 794 techniques. Les garder reviendrait à faire peser dans le calcul des mots qui ne distinguent rien, au point qu'une requête de pure grammaire comme « on or of » produirait un candidat. Elle ne produit maintenant aucun résultat, avec la note aucun terme significatif dans la requete.

Ensuite chaque mot reçoit un poids selon sa rareté dans le référentiel. Mesuré sur la version 19.2 : lsass pèse 5,89 et dump 4,12, tandis que credenti ne pèse que 2,55, window 2,35 et file 2,10. Un mot présent dans des centaines de techniques ne distingue rien.

Chaque technique est alors comparée à la requête sur six emplacements, du plus discriminant au moins discriminant : son nom et les acronymes définis dans sa description comptent pour 1,0 ; le nom de sa technique parente et les entités qui l'emploient — groupes, logiciels, campagnes — pour 0,8 ; les noms de ses tactiques pour 0,6 ; sa description pour 0,35.

Enfin chaque mot de la requête ne compte qu'une fois, au meilleur emplacement où il a été trouvé, et le total est rapporté au poids de la requête entière. Le score mesure donc la part de la requête retrouvée, entre 0 et 1. Sur notre exemple, le calcul se pose en entier :

poids requete : lsass 5,89 + credenti 2,55 + dump 4,12 = 12,55

T1003.001	LSASS Memory (parente : OS Credential Dumping)			
nom	lsass	1,00	$\times$ 5,89	= 5,89
parente	credenti, dump	0,80	$\times$ 6,67	= 5,34

		11,22	/ 12,55	= 0,894

T1003	OS Credential Dumping			
nom	credenti, dump	1,00	$\times$ 6,67	= 6,67
		6,67	/ 12,55	= 0,531

Le mot lsass est absent du nom *OS Credential Dumping* : c'est exactement ce qui fait passer la sous-technique devant sa parente ici, et le modèle peut le lire dans la réponse plutôt que de le supposer.

Deux plafonds corrigent les cas où cette couverture atteindrait 1 sans rien prouver. Le premier vise la requête d'un seul mot : « windows » se retrouve dans le nom de dizaines de techniques, et le mot unique de la requête y serait couvert en entier. Le score est donc plafonné à 0,59, juste sous le seuil fort, sauf si la requête est un identifiant vérifiable comme T1003.001 ou un nom strictement égal à celui d'une technique, comme « masquerading ». Le second vise la requête réduite au nom d'une entité : « mimikatz » désigne un logiciel, pas

un comportement, et les techniques qui remontent sont seulement celles que ce logiciel emploie. Elles sont plafonnées à 0,45 et marquées `matched.pivot`. Dès qu'une preuve lexicale accompagne l'entité, le plafond tombe : « `apt29 credential dumping` » rend bien ses candidats à 0,9.

Un nom trop court pour être un mot. Écarter les mots de moins de trois lettres a un effet de bord : la technique `T1053.002` s'appelle « `At` », deux caractères, et elle devenait introuvable par son propre nom. Plutôt que d'assouplir la règle et de réintroduire les mots vides, un index des noms exacts est construit au chargement, indépendant du découpage en mots. Il couvre le nom court et le nom complet Parente : Sous-technique. Résultat mesuré sur les 794 techniques interrogées par leur propre nom : aucune n'est introuvable, et 96,6 % sortent en première position.

Deux seuils sont rappelés dans chaque réponse : 0,6 pour un candidat fort, 0,3 en dessous duquel le serveur écrit lui-même que l'absence de mapping est une conclusion valide. À score égal, le départage est fixe et déterministe : nom exactement égal à la requête, puis nom le mieux couvert par la requête, puis technique parente avant sous-technique, puis matrice — Entreprise d'abord —, puis nom le plus court, puis identifiant. Deux exécutions de la même requête rendent donc la même liste dans le même ordre, condition pour qu'une analyse soit reproductible.

```
search_techniques("valid accounts")
$\\rightarrow$ T1078      Valid Accounts    1,0  Enterprise    nom égal, matrice IT
T0859      Valid Accounts    1,0  ICS           nom égal, matrice OT
T1078.003  Local Accounts    1,0  Enterprise    sous-technique, après
T1078.004  Cloud Accounts    1,0  Enterprise    sous-technique, après

"homonymes": [{"name": "Valid Accounts",
  "techniques": [{"id": "T1078", "matrix": "Enterprise"},
    {"id": "T0859", "matrix": "ICS"}]}
```

Le critère de matrice mérite qu'on s'y arrête, parce qu'il porte un risque propre à ce serveur. Vingt-six noms de techniques existent à l'identique dans Enterprise et dans ICS, et « `Valid Accounts` » en fait partie. À nom identique, le score l'est aussi, et il faut bien trancher d'une façon ou d'une autre : le classement place donc l'informatique de gestion en premier, faute de savoir dans quel monde travaille l'analyste. Le champ `homonymes` nomme l'ambiguïté quand elle se produit, et une note demande de préciser la matrice. Cela n'en fait qu'un amortisseur : le serveur ne connaît que le paramètre `matrix` qu'on lui transmet. Sur douze comportements typés, trois des premiers résultats tombaient dans la mauvaise matrice sans ce paramètre, et aucun avec. Le passer systématiquement est la règle.

Les preuves. Chaque candidat est accompagné des mots qui l'ont fait remonter et de l'em-

placement où ils ont été trouvés, dans le champ `matched`. Le modèle peut ainsi écarter le bruit avant même de consulter la fiche complète. Deux exemples mesurés montrent ce que cela apporte :

```
search_techniques("lateral movement rdp")
$> T1021.001 Remote Desktop Protocol 0,758
  matched: {"alias": ["rdp"], "tactic": ["later", "movement"]}
```

```
search_techniques("mimikatz")
$> T1098 Account Manipulation 0,45
  matched: {"software": ["S0002 Mimikatz"],
            "pivot": "entite seule : piste, pas un mapping"}
```

Dans le premier cas, `rdp` n'apparaît pas dans le nom de la technique : il vient de l'acronyme défini dans sa description, « Remote Desktop Protocol (RDP) », et les mots `lateral movement` ont été reconnus comme un nom de tactique. Le second cas montre l'inverse, une preuve qui n'en est pas une : aucun mot de la requête n'est dans le nom de la technique, c'est le logiciel *Mimikatz* qui a été reconnu, et les dix-huit techniques qui remontent sont simplement celles que ce logiciel emploie. Le champ `pivot` le dit en toutes lettres, le plafond de 0,45 empêche de les déclarer fortes, et une note renvoie à `get_software`, qui fait foi pour cette liste. Le champ `matched` dit donc dans les deux cas d'où vient le score, ce qui permet de juger sa solidité.

Ce que le moteur ne fait pas. Il ne comprend pas le sens des mots. Une requête en français ne trouve rien d'utile, et la réponse le dit :

```
search_techniques("imprimante thermique defectueuse")
$> 0 candidat
  note: aucun candidat : l'absence de mapping est une conclusion
        valide
```

```
search_techniques("vol d'identifiants dans la memoire")
$> T1591.004 Identify Roles 0,333
  note: aucun candidat MITRE fort : confronter le comportement au
        referentiel interne avec search_internal
```

Le second cas est le plus instructif : la requête décrit exactement le comportement recherché, et pourtant le meilleur candidat plafonne à 0,333 — et ce n'est même pas le bon, puisque c'est le mot `identifiants` qui a été rapproché de `Identify`. Aucun des mots français employés n'existe dans le catalogue. Les requêtes doivent donc être formulées dans le vocabulaire anglais d'ATT&CK, un comportement à la fois. C'est la règle que le modèle lit dans la description de l'outil avant d'appeler, et c'est aussi pourquoi la vérification par `get_technique`

reste obligatoire : un score élevé prouve que les mots correspondent, pas que le comportement est le bon.

2.3 ÉVALUATION DE L'APPROCHE

La méthodologie d'évaluation est celle du chapitre I, sans modification : même corpus, même étiquetage des prédictions, mêmes deux bases de calcul, mêmes formules de précision, de rappel et de F1, mêmes attributs de diagnostic et mêmes dénominateurs. Elle n'est pas reprise ici ; seuls les changements le sont. Un seul modèle est évalué, Claude Opus 4.8, retenu au terme de la sélection : la question n'est plus lequel choisir, mais ce que gagne celui qui a été choisi.

Une seconde question s'ajoute, qui n'a de sens qu'avec un serveur. Les métriques du chapitre I demandent *le mapping est-il juste ?* ; les indicateurs ajoutés ici demandent *la réponse vient-elle du référentiel ou de la mémoire du modèle ?* Un F1 qui progresse sans que la seconde soit posée ne prouverait rien : le modèle pourrait avoir mieux répondu sans jamais consulter la base.

2.3.1 La ligne à battre

Le tableau 2.2 rassemble les résultats de Claude Opus 4.8 sans serveur, sur les 42 échantillons et les 126 exécutions de la campagne du chapitre I. C'est le point de départ, et la seule référence contre laquelle la phase 2 se mesure.

	Base stricte	Base tolérante
F1 technique	0,563	0,571
F1 tactique	0,587	0,587
F1 sous-technique	0,636	0,682
F1 technique — Entreprise / ICS	0,714 / 0,262	—
Hors matrice · hallucination	0 · 0,0 %	0 · 0,0 %
Obsolescence · abstention	1,3 % · 0,0 %	1,3 % · 0,0 %

TABLEAU 2.2 – Claude Opus 4.8 sans serveur : la ligne à battre.

Hors qualité du mapping, la même campagne relève une stabilité de 76,2 %, une latence médiane de 4 282 ms pour un 95e centile de 6 050 ms, 6 572 jetons par exécution et une efficacité de 0,086 — le F1 technique obtenu par millier de jetons.

C'est la colonne tolérante qui fait référence. Les deux bases portent sur les mêmes réponses et ne diffèrent que sur un point : la base stricte compte comme une erreur un identifiant que

MITRE a retiré depuis, la base tolérante le crédite, puisque la table de correspondance publiée par MITRE atteste qu’il désignait bien le comportement attendu. Or résoudre ces identifiants retirés est exactement ce que le serveur mécanise. Comparer la phase 2 à la seule base stricte mélangerait donc deux effets : le rattrapage de version, que le serveur fait à coup sûr, et l’apport propre de la consultation. Le seuil à dépasser est donc **0,571 en technique, 0,587 en tactique et 0,682 en sous-technique**.

Deux chiffres ne peuvent pas s’améliorer. Ce modèle ne produit, sans serveur, aucun identifiant inventé ni aucun identifiant pris dans l’autre matrice : ces deux compteurs sont à zéro et ne peuvent donc que se dégrader. Leur dégradation serait un signal fort, car le serveur ne rend que des identifiants lus dans la base chargée — un identifiant inexistant apparaissant en phase 2 ne pourrait venir que de la mémoire du modèle, contre l’avis de l’outil qu’il vient d’interroger.

La marge se lit sur la matrice industrielle. Ventilé par matrice, le F1 technique passe de **0,714 sur Enterprise à 0,262 sur ICS**. L’écart n’est pas un accident de mesure : les comportements industriels sont moins documentés, moins présents dans les données d’entraînement, et le référentiel ICS a évolué plus récemment. C’est donc là qu’un accès à un référentiel à jour a le plus à apporter, et c’est sur cette ventilation que le gain se lira en premier.

2.3.2 Ce que la phase 2 ajoute au protocole

Une exécution n’est plus un appel et une réponse, mais une conversation : le modèle reçoit les journaux, demande un ou plusieurs outils, reçoit leurs résultats, et recommence jusqu’à pouvoir conclure. Quatre décisions en découlent.

- **L’appel d’outil est imposé au premier tour**, les tours suivants rendant la main. Sans cette contrainte, le modèle pourrait répondre de mémoire et l’exécution ne mesurerait pas l’ancrage mais son absence. **Conséquence à déclarer** : le taux d’utilisation du serveur vaut 100 % par construction et n’apprend donc rien. Ce sont le taux d’appels pertinents et le taux de réponses ancrées qui portent l’information.
- **Le modèle choisit ses outils**. Le serveur en publie dix-huit, et le modèle les découvre lui-même par *tools/list* : décider lequel appeler, et dans quel ordre, fait partie de ce que la campagne mesure. Restreindre le périmètre aux seuls outils utiles au mapping rendrait le taux d’appels pertinents égal à 100 % par construction, comme le taux d’utilisation. Une seule exclusion, qui n’est pas un choix de périmètre mais un garde-fou : *mitre_update* remplace la base en mémoire et changerait donc la version ATT&CK servie au milieu de la campagne. Les dix-sept autres sont en lecture seule.
- **La consigne système gagne un alinéa** : consulter le référentiel plutôt que répondre de mémoire, préciser la matrice, vérifier chaque candidat retenu, ne rien changer au

format de sortie. Le reste de la consigne est recopié sans modification, et les deux morceaux sont enregistrés séparément pour que la différence soit vérifiable. Coût méthodologique assumé : la comparaison mesure l’effet conjoint de l’accès au référentiel et de la consigne qui invite à s’en servir ; les séparer demanderait une troisième campagne, consigne retirée.

- **Le budget de tours du harnais**, fixé à huit allers-retours, s’ajoute aux cas déjà écartés du calcul. Une exécution qui n’y converge pas est close et écartée, au même titre qu’une réponse tronquée faute de jetons : ce sont deux limites du banc d’essai, pas deux échecs du modèle. L’effectif écarté est rapporté.

Le chapitre I compte en échantillons et en exécutions. S’y ajoute **l’appel d’outil** : une exécution en produit plusieurs, et compter en exécutions un comportement qui se répète dans la même conversation le sous-estimerait. Un indicateur portant sur la façon d’appeler se compte donc en appels, un indicateur portant sur la conduite d’une exécution se compte en exécutions.

E exécutions retenues C appels d’outils émis
T tours de conversation R appels de recherche ($R \leq C$)

2.3.3 Les indicateurs ajoutés

Onze indicateurs sont calculés depuis la trace que le harnais écrit pour chaque appel : l’outil invoqué, ses arguments, son issue, et les identifiants que le serveur a rendus. Aucun ne modifie un F1 ni n’entre dans un classement — ce sont des attributs de diagnostic, qui expliquent *comment* le modèle s’est servi du serveur là où les F1 mesurent ce qu’il en a tiré.

Indicateur	Formule	Unité
Taux d’utilisation	exécutions avec ≥ 1 appel / E	exécution
Appels par exécution	C / E	appel
Tours par exécution	T / E	tour
Taux d’appels pertinents	appels vers un outil rendant des identifiants / C	appel
Taux d’hallucination d’outil	appels vers un outil inexistant / C	appel
Appels en erreur	effectif brut — arguments refusés	appel
Démentis du serveur	effectif brut — identifiant déclaré introuvable	appel
Conformité de matrice	recherches portant la bonne matrice / R	appel

Indicateur	Formule	Unité
Exécutions conformes	exéc. sans recherche fautive / exéc. avec ≥ 1 recherche	exécution
Averti puis faux	effectif brut — homonymie signalée, identifiant hors matrice retenu	exécution
Taux de réponses ancrées	exéc. dont l'identifiant retenu a été rendu par un outil / exéc. ayant répondu	exécution

TABLEAU 2.3 – Les indicateurs ajoutés, leur formule et leur unité de compte.

Le taux d'appels pertinents vise les quatre outils dont la sortie porte des identifiants de tactique ou de technique. Les treize autres — mitigations, groupes, logiciels, campagnes, sources de journaux — ne sont pas fautifs : ils ne servent simplement pas cette tâche. L'indicateur mesure donc la pertinence de la *sélection* d'outil, ce qui suppose que le modèle ait eu le choix.

Le taux de réponses ancrées est le plus important des onze. Une réponse est dite ancrée lorsque l'identifiant finalement retenu figure parmi ceux que les outils ont effectivement rendus au cours de l'exécution. Dans le cas contraire, le modèle l'a produit de mémoire : pas nécessairement faux, mais non vérifié — or la vérification est précisément ce que le serveur devait apporter. Croisé avec l'étiquetage du chapitre I, cet indicateur sépare quatre situations que le seul F1 confondrait : ancrée et juste, l'ancrage a fonctionné ; ancrée et fausse, c'est le moteur de recherche qui a proposé le mauvais candidat ; non ancrée et juste, le modèle savait déjà et le serveur n'a rien apporté sur cet échantillon ; non ancrée et fausse, c'est exactement le cas que l'ancrage devait empêcher.

La conformité de matrice tranche entre deux causes d'un même symptôme. Le serveur ignore dans quel monde travaille l'analyste : il ne connaît que le paramètre *matrix* que le modèle lui transmet, et sans ce paramètre une recherche interroge les deux matrices à la fois. Comme vingt-six noms de techniques existent à l'identique dans Enterprise et dans ICS, un identifiant industriel peut alors être rendu pour un comportement bureautique, ou l'inverse. Un identifiant hors matrice subsistant en phase 2 admet donc deux explications opposées : ou bien le modèle n'a pas consulté le serveur, ou bien il l'a consulté sans lui dire dans quelle matrice chercher. La première se corrige par la consigne, la seconde par le harnais ; les confondre conduirait à corriger la mauvaise. Cette conformité est mesurée et non forcée : le harnais pourrait injecter la matrice dans chaque appel, mais tenir compte d'une contrainte énoncée dans sa consigne fait partie de ce que la campagne mesure.

Deux indicateurs existants changent de contenu, non de formule. Les **jetons** portent sur la somme de tous les tours d'une exécution, chacun rechargeant la consigne, les descriptions

d'outils et les résultats déjà reçus. La **latence** englobe les allers-retours ; la part imputable au serveur est isolée, et se compte en millisecondes quand l'exécution se compte en secondes — le surcoût de l'ancrage n'est donc pas le temps de réponse du référentiel, c'est le temps que le modèle passe à relire ce qu'il en a reçu.

2.3.4 Ce que la phase 2 doit montrer

Les attentes sont posées avant les résultats, pour qu'elles ne puissent pas être ajustées après coup.

- **Le taux d'obsolescence doit s'effondrer.** ATT&CK évolue à chaque version : des techniques sont retirées et remplacées par d'autres, et MITRE publie alors une relation de remplacement qui désigne l'identifiant vivant prenant la suite. La version 19.2 en compte 158. Un modèle dont les données d'entraînement sont antérieures à l'une de ces révisions continue de produire l'ancien identifiant : son raisonnement est juste, son numéro est périmé — c'est ce que mesurent les 1,3 % du tableau 2.2. Le serveur, lui, suit ces relations et ne rend jamais qu'un identifiant vivant. Un modèle qui consulte avant de répondre ne devrait donc plus en produire du tout. Si le taux ne baisse pas, c'est que la réponse n'est pas passée par le serveur, et le taux de réponses ancrées le confirmera sur l'exécution en cause.
- **Le gain doit se voir sur ICS avant tout.** L'écart de 0,714 à 0,262 entre les deux matrices désigne le terrain où la connaissance propre du modèle est la plus lacunaire. Un gain général et uniforme serait d'ailleurs suspect : il indiquerait un effet de consigne plutôt qu'un effet d'ancrage.
- **Les deux compteurs à zéro doivent le rester** — aucun identifiant inventé, aucun identifiant hors matrice.
- **Le coût doit être chiffré, pas éludé.** La conversation multiplie les jetons et allonge la latence. L'efficacité — le F1 technique obtenu par millier de jetons, 0,086 sans serveur — dira si ce surcoût achète de la qualité ou seulement du temps.

Le résultat qui invaliderait la lecture est un taux de réponses ancrées bas. Si le modèle appelle les outils parce que le harnais l'y oblige, puis répond de mémoire, les F1 mesureront sa connaissance du référentiel et non l'apport du serveur, et l'écart entre les deux phases perdrait son sens. C'est pourquoi cet indicateur est rapporté avant les F1, et non après.

2.3.5 Résultats

42 échantillons, 3 exécutions, 126 exécutions par condition. Aucune itération écartée : ni panne d'infrastructure, ni budget de sortie épuisé, ni boucle d'outils non convergente. Le serveur a servi la version 19.2 des deux matrices sur les 126 exécutions, sans dérive.

Niveau	MCP	N	VP	FP	H	O	A	MM
Tactique	non	126	74	52	0	0	0	0
Technique	non	126	71	54	0	1	0	0
Sous-technique	non	66	42	21	0	3	0	0
Tactique	oui	126	101	25	0	0	0	0
Technique	oui	126	89	37	0	0	0	0
Sous-technique	oui	66	50	16	0	0	0	0

TABLEAU 2.4 – Étiquetage des prédictions, sans puis avec serveur. N = VP + FP + H + O + A, vérifié sur les six lignes.

MCP	MM	Hall.	F1	F1	F1 s.-t.	Ent.	ICS	Obsol.	Abst.
			tech.	tact.					
non	0	0,0 %	0,563	0,587	0,636	0,714	0,262	1,3 %	0,0 %
oui	0	0,0 %	0,706	0,802	0,758	0,798	0,524	0,0 %	0,0 %

TABLEAU 2.5 – Qualité du mapping, base stricte. Ent. et ICS : F1 technique ventilé par matrice.

MCP	MM	Hall.	F1 tech.	F1 tact.	F1 s.-t.	Écart		Obsol.	Abst.
						obs.			
non	0	0,0 %	0,571	0,587	0,682	+0,008		1,3 %	0,0 %
oui	0	0,0 %	0,706	0,802	0,758	0,000		0,0 %	0,0 %

TABLEAU 2.6 – Qualité du mapping, base tolérante. Écart obs. = F1 tolérant - F1 strict, au niveau technique.

La référence à battre était 0,571 en technique, 0,587 en tactique et 0,682 en sous-technique. Les trois sont dépassées : **+0,135 en technique, +0,215 en tactique, +0,076 en sous-technique**. Le taux d'obsolescence tombe à zéro et l'écart entre les deux bases disparaît : il n'y a plus d'identifiant retiré à créditer. Aucun identifiant fabriqué ni hors matrice n'apparaît, de part et d'autre. Le gain est porté par la matrice industrielle, qui double presque — 0,262 à 0,524 —, tandis qu'Enterprise progresse de 0,084 : le gain n'est ni général ni uniforme, ce qui écarte l'hypothèse d'un simple effet de consigne. Deux diagnostics achèvent la lecture : les 9 confusions de la scission v19 disparaissent complètement, et les sous-techniques niées à tort passent de 9 à 6.

L'obsolescence ne diminue pas : elle disparaît. Le taux d'obsolescence ne baisse pas : il disparaît. Les quatre identifiants retirés que Claude Opus 4.8 produisait sans serveur prove-

naient de deux échantillons seulement, et aucun des deux n'en produit plus. Sur S027 — effacement des journaux système Linux —, le modèle répondait T1070.002 aux trois exécutions, un identifiant révoqué au profit de T1685.006 qui porte exactement le même nom ; avec le serveur, les trois exécutions rendent T1685.006. Sur S007 — message Modbus non autorisé —, il répondait T0836 deux fois et T0855 une fois, ce dernier révoqué au profit de T1692.001 ; avec le serveur, les trois exécutions rendent T1692.001. Le troisième échantillon du corpus exposé à une révocation, S038, était déjà juste sans serveur et le reste.

Éch.	Matrice	Sans serveur	Avec serveur
S027	Enterprise	T1070 / T1070.002 (trois exécutions)	T1685 / T1685.006 (trois exécutions)
S007	ICS	T0836 ×2, puis T0855 (une exécution)	T1692 / T1692.001 (trois exécutions)
S038	Enterprise	T1547 / T1547.001 (déjà juste)	T1547 / T1547.001 (inchangé)

TABEAU 2.7 – Les échantillons du corpus exposés à une révocation, avant et après ancrage.

Ce que ces deux cas démontrent. Ces deux corrections ne relèvent pas d'un meilleur raisonnement. Sur S027, le modèle avait déjà identifié la bonne action — effacer des journaux — et nommait la bonne technique sous son ancien numéro ; il lui manquait de savoir que ce numéro avait changé. Sur S007, le cas est plus net encore : deux exécutions sur trois donnaient T0836, une technique vivante mais fausse, que la base tolérante ne rattrape pas. L'ancrage corrige donc deux défauts distincts avec le même mécanisme — un référentiel périmé et une confusion ordinaire — parce que dans les deux cas le modèle a consulté avant de répondre au lieu de puiser dans sa mémoire.

Portée du résultat. C'est le résultat le plus net de la campagne, et le plus directement attribuable au serveur. Un identifiant retiré est un défaut que le modèle ne peut pas corriger seul : rien dans le journal ne lui signale que le référentiel a bougé depuis son entraînement. Le serveur, lui, suit les relations de remplacement publiées par MITRE et ne rend jamais qu'un identifiant vivant. Le passage de 1,3 % à 0 % n'est donc pas une amélioration graduelle mais la suppression d'une classe entière d'erreurs.

Util.	App./ex.	Tours/ex.	Pertin.	Hall. outil	Ancrées	Matrice	Vérif.
100 %	2,9	3,4	100 %	0,0 %	100 %	100 %	100 %

TABLEAU 2.8 – Usage du serveur, campagne avec MCP.

Serveur rend + modèle retient	Serveur rend, modèle retient autre	Serveur ne rend pas, modèle juste	Serveur ne rend pas, modèle faux
70,6 %	19,0 %	0,0 %	10,3 %

TABLEAU 2.9 – Croisement, au niveau technique, entre ce que le serveur a rendu et ce que le modèle a retenu.

Le modèle a appelé un outil à chaque exécution, 2,9 fois en moyenne sur 3,4 tours, sans jamais inventer d'outil, se tromper d'argument ni omettre la matrice sur les 163 recherches. Il a vérifié chaque candidat retenu par *get_technique*, et huit appels ont déclenché la redirection d'un identifiant révoqué. Trois outils sur les dix-sept exposés ont été employés : *get_technique* 184 fois, *search_techniques* 163 fois, *get_tactic* 22 fois.

Le taux d'ancrage de 100 % valide la lecture des tableaux précédents : aucune réponse n'a été produite de mémoire, l'écart mesure donc bien l'apport du serveur. Le croisement situe la marge restante : dans **19,0 %** des exécutions le serveur avait rendu la bonne technique et le modèle en a retenu une autre — échec de sélection, corrigible par la consigne ; dans **10,3 %** le serveur ne l'a jamais rendue — échec du moteur ou de la requête formulée. Aucune bonne réponse n'a été produite sans que le serveur l'ait rendue.

MCP	Stabilité	p50	p95	Jetons/exéc.	Sur-spéc.	Efficacité
non	76,2 %	4 282 ms	6 050 ms	6 572	10,0 %	0,086
oui	71,4 %	14 146 ms	22 554 ms	47 441	18,3 %	0,015

TABLEAU 2.10 – Fiabilité, vitesse et coût.

Le prix de l'ancrage est élevé : **7,2 fois plus de jetons** et **3,3 fois la latence médiane**, pour 3,4 tours dont chacun recharge la consigne, les descriptions d'outils et les résultats déjà reçus. L'efficacité passe de 0,086 à 0,015. La stabilité recule de 4,8 points, la boucle d'outils ouvrant des chemins multiples vers une même conclusion, et la sur-spécification augmente — le modèle, mieux informé des sous-techniques existantes, en affirme davantage sur des techniques terminales.

2.4 LES AVANTAGES DE L'APPROCHE

Quatre bénéfices découlent de cette architecture. Les trois premiers portent sur la qualité du mapping, le quatrième sur le déploiement.

Le versionnage du référentiel. C'est l'avantage principal. ATT&CK n'est pas figé : des techniques apparaissent, d'autres fusionnent ou sont retirées à chaque révision. Un modèle qui répond de mémoire restitue un état du référentiel qu'il ne sait pas dater, et l'analyste n'a aucun moyen de le vérifier. Notre serveur sert au contraire une version nommée, choisie et épinglée, et la joint à chaque réponse : une analyse est datée, et rejouable à l'identique des mois plus tard. Les identifiants retirés ne sont pas perdus pour autant, puisqu'un identifiant révoqué est redirigé vers son remplaçant vivant — T1086, par exemple, conduit à T1059.001. Un rapport ancien reste donc exploitable sans travail de conversion. Sur notre campagne d'évaluation, le taux d'identifiants obsolètes est passé de **1,3 %** à **0 %**.

La personnalisation interne. ATT&CK décrit des comportements observés à l'échelle mondiale ; une organisation en observe qui lui sont propres, ou que le référentiel public ne couvre qu'avec une granularité insuffisante. Le référentiel interne permet de les décrire sous des identifiants maison, alimentés par les rapports de red team et les productions de renseignement internes, chaque fiche demeurant rattachée à une technique ATT&CK (section 2.2.2). Les deux niveaux cohabitent sans se contredire : la précision locale est ajoutée là où le référentiel public s'arrête, et le vocabulaire commun reste intact, de sorte qu'un interlocuteur externe comprend toujours de quoi il est question.

La précision maîtrisée. Le serveur ne rend pas le modèle plus savant : il rend ses réponses vérifiables. Tout identifiant renvoyé provient de la base chargée, ce qui retire au modèle la possibilité d'en composer un qui n'existe pas. Chaque candidat arrive accompagné des indices qui l'ont fait remonter, et l'absence de correspondance constitue une réponse explicite plutôt qu'un identifiant forcé. L'effet est mesurable : sur la même campagne, le score F1 du mapping est passé de **0,571** à **0,706**.

La polyvalence. Le serveur ignore qui l'appelle : il parle MCP, et tout client qui parle MCP peut s'y brancher. Changer d'usage revient donc à changer de client, sans rien modifier au serveur. Le premier cas d'utilisation en donne la mesure : une chaîne conversationnelle bâtie dans n8n (figure 2.7). La question de l'analyste entre par un nœud de discussion et parvient à un agent porté par Claude Sonnet, auquel sont rattachés une mémoire de session et le serveur MCP MITRE déclaré comme outil. S'y ajoute une intelligence procédurale : un fichier de compétences décrit à l'agent la marche à suivre, c'est-à-dire la méthode de mapping elle-même. Chaque appel est ensuite assemblé puis transmis par HTTP au service de journalisation, avant que la réponse ne revienne dans le fil, sourcée et datée. L'ensemble se comporte comme un assistant d'analyste : il dispose du référentiel public et des fiches internes, et répond à toute question posée dans la conversation.

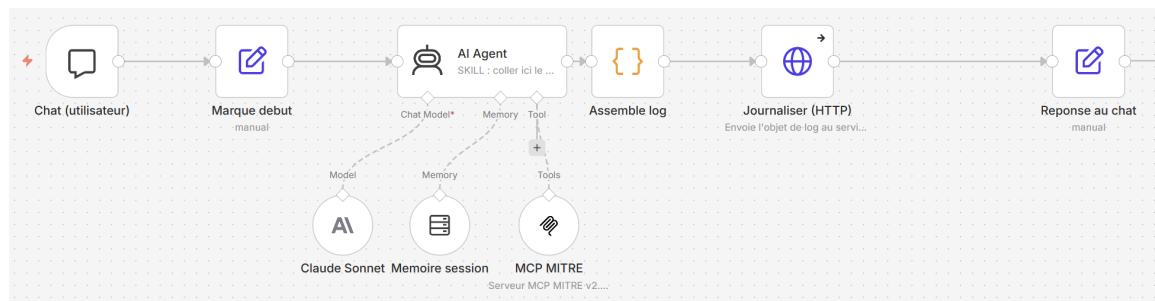


FIGURE 2.7 – Cas d’utilisation : chaîne conversationnelle n8n. L’agent reçoit la question, dispose du serveur MCP MITRE comme outil, et chaque appel est journalisé avant le retour de la réponse.

CONCLUSION GÉNÉRALE

Cet essai partait d'un constat : le goulot d'étranglement d'un centre d'opérations de sécurité n'est pas la détection, mais la qualification de ce qui est détecté. Il a conçu et mesuré un écosystème ancrant un modèle de langage sur MITRE ATT&CK par le *Model Context Protocol*.

Les modèles seuls ne suffisent pas, et l'écart se creuse côté industriel. Sur 42 échantillons produits en laboratoire, hors contamination, dix modèles courants ont été évalués en 1 260 appels. Aucun ne dépasse 0,603 de F1 technique en base stricte et les cinq premiers se tiennent en moins de 0,04 : il n'y a pas de vainqueur net. La ventilation par matrice est plus instructive que le classement — le meilleur passe de 0,679 en Entreprise à 0,452 en ICS. Les vingt-trois identifiants fabriqués du panel sont tous des tactiques, et tous sur des journaux industriels : l'angle mort signalé par la revue est mesuré, et il est d'abord un défaut de repérage entre univers.

L'ancrage corrige ce que le modèle ne peut pas corriger seul. À corpus et conditions identiques, le modèle retenu progresse de 0,563 à 0,706 en technique et de 0,587 à 0,802 en tactique. Le gain n'est pas uniforme : la matrice industrielle double presque, de 0,262 à 0,524, ce qui écarte l'hypothèse d'un effet de consigne. Surtout, le taux d'identifiants obsolètes ne diminue pas, il disparaît — une classe entière d'erreurs supprimée, qu'aucun raisonnement ne rattrape puisque rien dans le journal ne signale que le référentiel a bougé depuis l'entraînement.

Le mérite revient à la conception du serveur, non au simple accès à une source : une version nommée et jointe à chaque réponse, la résolution des révocations, une granularité d'outils bornée, une trace d'usage vérifiable. Le risque, lui, est déplacé et non supprimé : le serveur devient un canal d'instruction autant qu'une source de faits, et l'audit de cette surface reste à mener.

Trois limites bornent ces résultats : un seul modèle évalué avec ancrage, un corpus de 42 échantillons, et un effort de raisonnement laissé aux valeurs par défaut faute d'échelle commune entre éditeurs. Reste l'enseignement principal : les modèles les plus capables sont les plus exposés, parce que l'attaque exploite l'aptitude même qui fait leur valeur. La réponse n'est pas un modèle plus grand, mais un cadre qui ancre, borne et journalise.

BIBLIOGRAPHIE

- [1] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model Context Protocol (MCP) : Landscape, security threats, and future research directions. arXiv :2503.23278, 2025.
- [2] Zhiqiang Wang, Yichao Gao, et al. MCPTox : A benchmark for tool poisoning attack on real-world MCP servers. In *Proc. AAAI Conference on Artificial Intelligence*, 2026.
- [3] Or Peles. CVE-2025-6514 : Critical RCE vulnerability hijacking mcp-remote clients (CVSS 9.6). Avis de sécurité, JFrog Security Research, 2025.
- [4] Avi Lumelsky. CVE-2025-49596 : Critical remote code execution in the Anthropic MCP Inspector (CVSS 9.4). Avis de sécurité, Oligo Security ; fiche NVD, 2025.
- [5] Anthropic. Introducing the Model Context Protocol. Billet officiel, 25 novembre 2024, 2024. URL <https://www.anthropic.com/news/model-context-protocol>.
- [6] Model Context Protocol. Specification – changelogs 2025-03-26 et 2025-06-18. Autorisation OAuth 2.1, Streamable HTTP, annotations d’outils ; *resource servers*, RFC 8707, sortie structurée, *elicitation*, bonnes pratiques de sécurité, 2025. URL <https://modelcontextprotocol.io/specification/2025-06-18/changelog>.
- [7] Model Context Protocol. Specification 2026-07-28. Billet de publication « The 2026-07-28 Specification » par D. Soria Parra et D. Delimarsky, 28 juillet 2026, 2026. URL <https://modelcontextprotocol.io/specification/2026-07-28>.
- [8] [auteur à compléter]. [titre à compléter — enquête citée dans l’introduction générale], 2024. Référence à compléter.
- [9] Shahroz Tariq, Mohan Baruwat Chhetri, Surya Nepal, and Cecile Paris. Alert fatigue in security operations centres : Research challenges and opportunities. *ACM Computing Surveys*, 57(9) :224 :1–224 :38, 2025.
- [10] Blake E. Strom, Andy Applebaum, Doug P. Miller, Kathryn C. Nickels, Adam G. Pennington, and Cody B. Thomas. MITRE ATT&CK : Design and philosophy. Technical Report MP180360R1, The MITRE Corporation, 2020. Première édition 2018, révisée 2020.

- [11] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12) :248 :1–248 :38, 2023.
- [12] Anthropic and Linux Foundation. Donating the Model Context Protocol and establishing the Agentic AI Foundation. Billet officiel d’Anthropic et communiqué de la Linux Foundation, 9 décembre 2025, 2025.
- [13] Hamza Kheddar. Transformers and large language models for efficient intrusion detection systems : A comprehensive survey. *Information Fusion*, 2025.
- [14] Center for Threat-Informed Defense (MITRE Engenuity). TRAM – threat report ATT&CK mapper. Dépôt logiciel, 2024. URL <https://github.com/center-for-threat-informed-defense/tram>.
- [15] Md Rayhanur Rahman, Rezvan Mahdavi-Hezaveh, and Laurie Williams. What are the attackers doing now ? Automating cyberthreat intelligence extraction from text on pace with the changing threat landscape : A survey. *ACM Computing Surveys*, 55(12), 2023.
- [16] Zhenyuan Li, Jun Zeng, Yan Chen, and Zhenkai Liang. AttackKG : Constructing technique knowledge graph from cyber threat intelligence reports. In *Computer Security – ESORICS 2022*, volume 13554 of *Lecture Notes in Computer Science*, pages 589–609. Springer, 2022.
- [17] Md Tanvirul Alam, Dipkamal Bhusal, Le Nguyen, and Nidhi Rastogi. CTIBench : A benchmark for evaluating LLMs in cyber threat intelligence. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.
- [18] Md Tanvirul Alam, Dipkamal Bhusal, Salman Ahmad, Nidhi Rastogi, and Peter Worth. AthenaBench : A dynamic benchmark for evaluating LLMs in cyber threat intelligence. In *Workshop on AI for Cyber Threat Intelligence (WAITI)*, 2025. arXiv :2511.01144.
- [19] Manish Bhatt, Sahana Chennabasappa, Cyrus Nikolaidis, Shengye Wan, Ivan Evtimov, et al. Purple Llama CyberSecEval : A secure coding benchmark for language models. arXiv :2312.04724 ; et CyberSecEval 2, arXiv :2404.13161 (2024), 2023.
- [20] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. Vulnerability detection with code language models : How far are we ? In *Proc. 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 469–481, 2025.

- [21] Emanuele Mezzi, Fabio Massacci, and Katja Tuma. Large language models are unreliable for cyber threat intelligence. *arXiv :2503.23175*, 2025.
- [22] Ian R. McKenzie et al. Inverse scaling : When bigger isn't better. *Transactions on Machine Learning Research (TMLR)*, 2023.
- [23] Weijie Kong, Ahmed M. Saber, Amr Youssef, and Deepa Kundur. Large language models as explainable cyberattack detectors for energy industrial control systems. In *ACM EnergySP 2026 (co-localisé avec ACM e-Energy)*, 2026. *arXiv :2604.26079*.
- [24] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.
- [25] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jia-wei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models : A survey. *arXiv :2312.10997*, 2023.
- [26] Ahmed Lekssays et al. TechniqueRAG : Retrieval-augmented generation for adversarial technique annotation in cyber threat intelligence text. *arXiv :2505.11988*, 2025.
- [27] Prasanna N. Wudali et al. Rule-ATT&CK mapper (RAM) : Mapping SIEM rules to TTPs using large language models. *arXiv :2502.02337*, 2025.
- [28] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. PoisonedRAG : Knowledge corruption attacks to retrieval-augmented generation of large language models. In *34th USENIX Security Symposium*, pages 3827–3844, 2025.
- [29] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer : Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
- [30] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct : Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [31] Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, et al. Tool learning with foundation models. *ACM Computing Surveys*, 57(4) :101 :1–101 :40, 2024.

- [32] Montimage. mitre-mcp – a Model Context Protocol (MCP) server for the MITRE ATT&CK framework. Dépôt logiciel, 2025. URL <https://github.com/montimage/mitre-mcp>.
- [33] Anthropic and Verizon. Mapping AI-enabled cyber threats to MITRE ATT&CK. Rapport technique, 2026. URL <https://red.anthropic.com/2026/attack-navigator/>.
- [34] Tiantian Gan and Qiyao Sun. RAG-MCP : Mitigating prompt bloat in LLM tool selection via retrieval-augmented generation. arXiv :2505.03275, 2025.
- [35] Xiang Fei, Xiawu Zheng, and Hao Feng. MCP-Zero : Active tool discovery for autonomous LLM agents. arXiv :2506.01056, 2025.
- [36] Ziyang Luo, Zhiqi Shen, Wenzhuo Yang, Zirui Zhao, Prathyusha Jwalapuram, Amrita Saha, Doyen Sahoo, Silvio Savarese, Caiming Xiong, and Junnan Li. MCP-Universe : Benchmarking large language models with real-world Model Context Protocol servers. arXiv :2508.14704 ; Salesforce AI Research, 2025.
- [37] Yixuan Yang, Daoyuan Wu, and Yufan Chen. MCPSecBench : A systematic security benchmark and playground for testing Model Context Protocols. arXiv :2508.13220, 2025.
- [38] OWASP Foundation. OWASP top 10 for LLM applications (2025) et OWASP top 10 for agentic applications (2026). Classements de référence, 2026.
- [39] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for : Compromising real-world LLM-integrated applications with indirect prompt injection. In *16th ACM Workshop on Artificial Intelligence and Security (AISec), ACM CCS*, pages 79–90, 2023.
- [40] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent : Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics (ACL)*, 2024.
- [41] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo : A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

- [42] Invariant Labs. MCP security notification : Tool poisoning attacks. Billet technique, avril 2025 (littérature grise ; première divulgation publique de l’attaque), 2025.
- [43] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. Model Context Protocol (MCP) at first glance : Studying the security and maintainability of MCP servers. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 2026. doi :10.1145/3814959; arXiv :2506.13538.
- [44] Brandon Radosevich and John Halloran. MCP safety audit : LLMs with the Model Context Protocol allow major security exploits. arXiv :2504.03767 (prépublication), 2025.
- [45] National Security Agency, Artificial Intelligence Security Center (AISC). Model Context Protocol (MCP) : Security design considerations for AI-driven automation. Cybersecurity Information Sheet U/OO/6030316-26, National Security Agency, mai 2026.
- [46] MITRE. ATLAS – adversarial threat landscape for artificial-intelligence systems. Base de connaissances, 2026. URL <https://atlas.mitre.org>.
- [47] Parlement du Canada. Projet de loi c-8, loi concernant la cybersécurité, modifiant la loi sur les télécommunications et apportant des modifications corrélatives à d’autres lois (partie 2 : Loi sur la protection des cybersystèmes essentiels). 45^e législature, 1^{re} session ; sanction royale, juin 2026, 2026.
- [48] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. PentestGPT : Evaluating and harnessing large language models for automated penetration testing. In *33rd USENIX Security Symposium*, pages 847–864, 2024.